影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 18 章 期末專題：4P 實踐

本章學習目標

- 闡述 4P 創意學習(Projects、Passion、Peers、Play)的理念與其在工程教育中的意義。
- 說明創意學習螺旋的五個階段，並以之規劃專題的迭代開發流程。
- 選定一個兼具熱情與可行性的專題題目，並拆解為可執行的里程碑。
- 運用本書十七章的技術，整合出一個端到端的視覺應用系統。
- 進行有效的團隊協作、版本管理與技術文件撰寫。
- 完成專題成果的展示、反思與評估，並規劃後續的學習路徑。

叫獸開講|終身幼兒園：千年來最重要的發明是什麼?

世紀之交，一場研討會上有人拋出了一個問題：過去一千年來，最重要的發明是什麼?與會者陸續給出答案——有人說是印刷術，讓知識得以大量流通;有人說是蒸汽機，啟動了工業革命;有人說是電燈泡，把人類的活動延伸到夜晚。輪到麻省理工學院媒體實驗室的 Mitchel Resnick 教授時，他給了一個讓全場愣住的答案幼兒園。

這聽起來像個玩笑，但 Resnick 是認真的。他的理由是：一百多年前誕生的幼兒園，是人類教育史上一次根本的創新——它不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解。孩子們用積木蓋城堡、用蠟筆畫故事、在遊戲中學會協商與分享。這個過程，正是創造力的搖籃。

Resnick 觀察到一個令人遺憾的現象：離開幼兒園之後，學校教育就變了樣——孩子開始長時間坐在課桌前，填寫習題、聽講、應付考試。更糟的是，近年連幼兒園本身都在「小學化」，積木與手指畫被數學練習卷與識字卡取代。他的主張因此格外鮮明：不該讓幼兒園變得像其他學校，而該讓其他學校——甚至整個人生——變得更像幼兒園。

這不是要大人去玩積木，而是要保留幼兒園那套學習方式的精髓。Resnick 帶領的研究群就叫「終身幼兒園」(Lifelong Kindergarten)，他們把這套精髓提煉成四個以 P 開頭的原則：專題(Projects)、熱情(Passion)、同儕(Peers)、遊玩(Play)——讓人們基於自己的熱情、與同儕協作、以遊玩的精神，投入專題的創作。這個研究群的成果你可能早就用過：他們開發了全球最大的兒童程式學習平台 Scratch，也與樂高合作催生了 LEGO Mindstorms 機器人套件。Resnick 的學術血脈同樣耀眼——他的博士指導教授是建構主義學習理論的先驅 Seymour Papert。

而這，正是這門課從第一章走到現在的隱藏主線。回頭看看：每一章開頭的「叫獸開講」，講的都是一個真實的人、在真實的處境中，因為某種熱情或不甘

而做出的創造——傅立葉在行政庶務之餘寫下被退稿的論文、張正友用一張紙取代昂貴的標定物、Muybridge 為了一場關於馬蹄的爭論架起一整排相機、Redmon 在成就的頂點選擇放下、樂鑫的一群工程師讓五美元的晶片改變了創客世界。這些故事不是花絮，它們是想告訴你：技術從來不是冷冰冰的公式，而是活生生的人，帶著熱情與判斷，一步步做出來的。

現在，輪到你了。期末專題不是一份要交出去的作業，而是這門課給你的一次機會：去做一個你真心想做的東西，和你的夥伴一起，帶著玩的心情，把這十七章學到的技術變成一個真實存在的作品。做出來之後，它會是你的——不是老師的、不是課本的，是你的。

思考題：回想你的求學歷程，哪一次學習經驗讓你最投入、學得最深刻?它符合 4P 中的哪幾項?為什麼那次經驗與「為了考試而讀書」感覺如此不同?

18.1 4P 創意學習

4P 是本課程的教學設計骨架，也是期末專題的評量精神。理解它，你才會明白為什麼這門課要這樣上、專題要這樣做。



這是一張說明圖，不是實測。4P 是 18.1 節的四個原則，而它們並不是期末才登場的口號——本書從第 1 章開始就按這個骨架設計：每一章都以 Lab 收尾（Projects）、每一章的叫獸開講講的都是某個人的熱情（Passion）、每一章都要求你把結果與同學比對（Peers）、而幾乎每一個 Lab 都在要求你試參數、看結果、再調整（Play）。這四根柱子裡最常被低估的是熱情，因為它決定了你願意為一個卡住的問題坐多久;最常被誤解的是遊玩，它不是不認真，而是「投入來自好奇而非壓力」。接下來的每一張圖，都會把其中一根柱子換算成可以量的東西。

圖 18-1　4P：這門課其實一直在用的教學骨架

18.1.1 Projects：專題

人們在動手創造具體事物的過程中學得最深。聽講與寫習題能傳遞資訊，但真正的理解來自於「試著做出來卻卡住，然後想辦法解決」的那個當下。當你為了讓機器人認出障礙物而反覆調整參數、當你發現訓練出來的模型在教室裡很準、拿到走廊就

失靈——那些挫折時刻學到的東西，遠比課本上的段落深刻。這也是為什麼本書每一章都有 Lab，而整門課以專題收尾：知識要透過創造才能真正內化。

18.1.2 Passion：熱情

這是 4P 中最容易被忽略、卻最關鍵的一項。當人們投入自己真正在乎的題目時，願意付出更多時間、承受更多挫折、學得更多東西。所以選題時，叫獸的建議永遠是：選一個你自己會想用、會想給家人朋友看的東西。是替阿嬤的果園做病蟲害偵測?是幫社團管理器材借還?是做一個能陪你打電動的機器人?題目不必偉大，但必須是你的。這裡要補一個很實際的理由，而不只是勵志的說法：熱情決定了你願意為一個卡住的問題坐多久，而專題的進度幾乎都是在那些時刻產生的。圖 18-3 量過一個例子：同一條零件計數的管線，照抄預設參數時完全數對的比例只有 8%;五輪調整之後是 90%。那五輪不難，但每一輪都要先看懂上一輪為什麼失敗——而願意去看懂，就是熱情在工程上的樣子。反過來說，一個你不在乎的題目，你會在第 0 輪那個難看的數字出現時就想換題目。

請特別注意：熱情不是「一開始就有」，更多時候是「做著做著長出來的」。若你暫時想不到有熱情的題目，就先動手做一個看起來有趣的，過程中的小成就感會慢慢點燃它。

18.1.3 Peers：同儕

創造很少是孤獨的事。與同伴協作、互相給予回饋、分享彼此的發現，能讓每個人走得更遠。專題採分組進行，不只是為了分攤工作量，更是因為：當你必須向隊友解釋你的做法時，你會發現自己理解上的漏洞;當隊友提出你沒想過的角度時，你的方案會變得更好。

同儕也不限於同組隊友。向其他組請教、參考開源社群的做法、在網路上提問——這些都是同儕學習。承第 13 章 ORB 的故事、第 17 章樹莓派的故事：整個開源與創客文化，本身就是「同儕協作」在全球尺度上的展現。

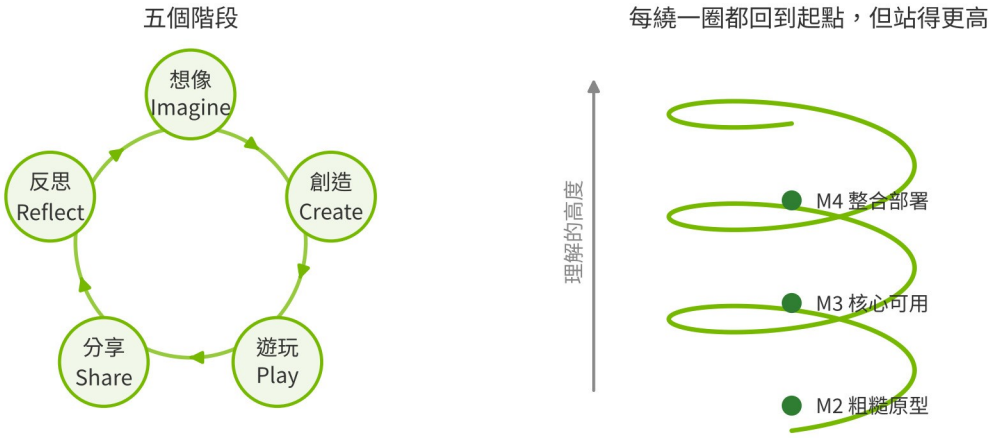
18.1.4 Play：遊玩

遊玩的精神，指的是願意嘗試、不怕失敗、樂於實驗的心態。它與「不認真」正好相反——遊玩需要高度的投入，只是這份投入來自好奇而非壓力。在專題中，這意味著：大膽試試看那個不確定會不會成功的想法、把出錯的結果當成有趣的線索而非災難、在做出基本功能後想想「如果加上這個會怎樣」。

工程上，遊玩的精神有一個實際的名字叫「快速原型」：先做一個粗糙但能動的版本，從中學習，再改進。這比在紙上完美規劃三個月然後一次做對，更符合真實的創造歷程。

18.2 創意學習螺旋

Resnick 觀察幼兒園裡孩子玩積木的過程，提煉出一個循環，稱為創意學習螺旋——這也是專題開發最貼近真實的流程模型：



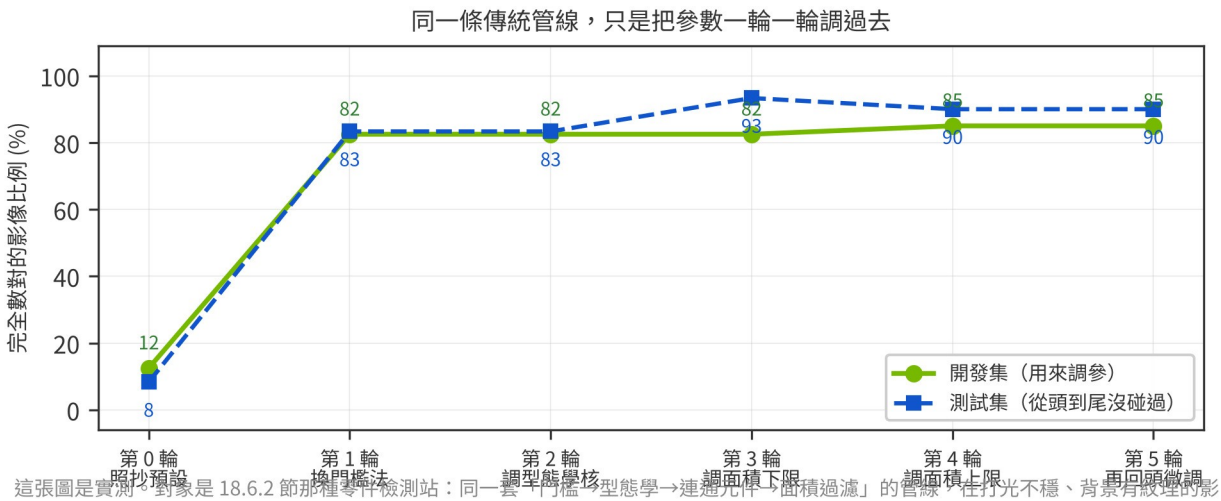
這也是一張說明圖。左邊是 18.2 節的五個階段，右邊說明「螺旋」這個字的意思：你確實一再回到「想像」，但每一次回來，手上有的不再是上一次的那個想法——你已經知道哪裡會卡、哪個假設是錯的、哪條路不必再試。這就是為什麼 18.3.2 節的里程碑要切成好幾段：一個里程碑就是螺旋的一圈，而每一圈都必須有「能動的東西」可以玩、可以給別人看，否則就沒有東西可以反思。很多專題的失敗不是因為技術不夠，而是因為整學期只繞了一圈——最後兩週才第一次把東西跑起來，於是沒有第二次機會。下一張圖會把「多繞幾圈真的有用嗎」這件事量出來。

圖 18-2　創意學習螺旋：為什麼是螺旋而不是圓圈

1. 想像(Imagine)：產生一個想法——「我想做一個能自動辨識回收物的垃圾桶」。
2. 創造(Create)：動手把它做出來，哪怕只是最粗糙的版本。
3. 遊玩(Play)：實際玩玩看、測試它，在各種情況下試探它的行為。
4. 分享(Share)：給別人看、聽取回饋——同學、老師、家人的反應往往能點出你沒發現的問題。
5. 反思(Reflect)：想想哪裡好、哪裡不好、為什麼——然後回到「想像」，展開下一輪。

之所以稱為「螺旋」而非「循環」，是因為每繞一圈，你都回到起點但站在更高的位置：你的想法更成熟、技術更純熟、對問題的理解更深。這正是專題應有的節奏——不要期待一次做對，而要規劃多次迭代。

這個螺旋與工程界的敏捷開發、與您在本書各章 Lab 中反覆經歷的「試參數、看結果、再調整」，骨子裡是同一件事。而它也解釋了為什麼 18.3 節的專題時程要切成多個里程碑：每個里程碑就是螺旋的一圈。「多繞幾圈真的有用嗎」這件事可以量。圖 18-3 拿本章那個零件計數的任務跑了五輪：第 0 輪照抄範例的預設參數，測試集上完全數對的比例是 8%;第 1 輪把全域門檻換成自適應門檻，跳到 83%;五輪之後是 90%。這張圖還順便說明了兩件不太中聽的事。第一，幾乎所有的進步都來自同一個改動——後面四輪加起來只移動了幾個百分點。卡住的時候該問的往往是「我這一步的假設對不對」，而不是「這個數字再調大一點會不會好一些」。第二，最後兩輪在開發集上還在進步，測試集卻退了一點點;你每在開發集上挑一次最好的參數，就把一點點開發集的運氣寫進了系統裡。所以螺旋要繞，但每一圈的成績要報在一份從頭到尾不碰的測試集上。



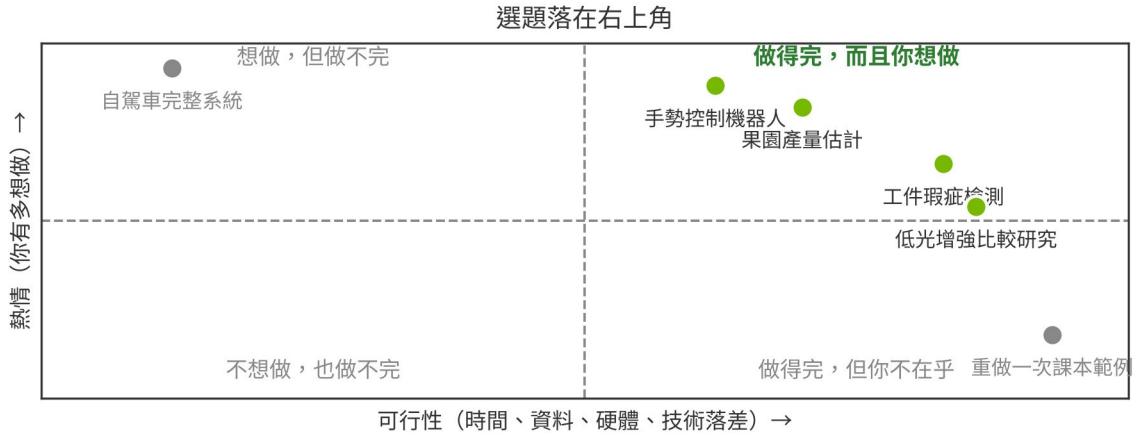
這張圖是實測，對象是 18.6.2 節那種零件檢測站：同一套「門檻→型態學→連通元件→面積過濾」的管線，在打光不穩、背景有紋理的影像上數零件有幾個。第 0 輪直接用範例的預設參數，完全數對的比例只有 8%——也就是幾乎每一張都數錯。接下來每一輪只動一組參數（這正是真實的調參過程，一次動太多你會不知道是哪一個起的作用），在開發集上挑最好的那個值，再拿到測試集上驗。五輪之後測試集是 90%。這就是螺旋的工程版本：每一圈的改進都來自上一圈量到的結果，而不是憑空的靈感。但這張圖真正值得細看的是兩件事，而且都有點不中聽。第一，幾乎所有的進步都來自同一個改動。第 1 輪把全域門檻換成自適應門檻，測試集就從 8% 跳到 83%;後面四輪加起來只再移動了幾個百分點。這是調參很常見的形狀——卡住時該問的往往是「我這一步的方法對不對」，而不是「這個數字再調大一點會不會好一些」。第二，最後兩輪在開發集上從 82% 進步到 85%，測試集卻從 93% 退到 90%。你在開發集上每挑一次最好的參數，就把一點點開發集的運氣寫進了系統裡（第 14 章的過度擬合，只是這次過擬的是你的手而不是模型）。不過也請不要過度解讀這幾個百分點：測試集只有 60 張，在九成上下的水準，它的標準誤就有三到四個百分點（推導 18.5.3 會把這條式子寫出來），所以 93% 與 90% 根本分不出高下。這正是為什麼專題要留一份從頭到尾不碰的測試集，而且報告裡要一併寫出它有幾張。

圖 18-3　多繞幾圈真的有用嗎：五輪調參的實測

18.3 專題的規劃與執行

18.3.1 選題：熱情與可行性的交集

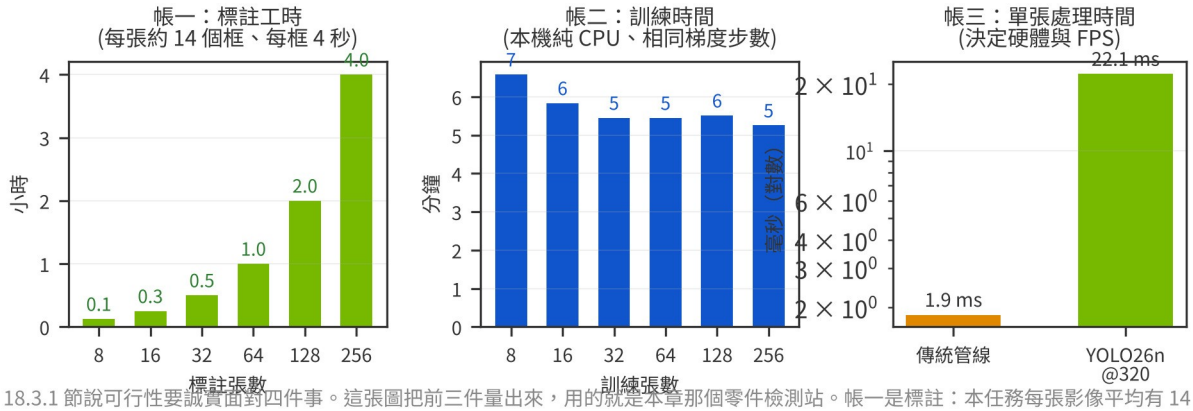
好的題目落在兩個圓的交集：一邊是「你真心想做」(Passion)，另一邊是「在時間與資源內做得出來」(可行性)。只有熱情而不可行，會做到一半崩潰;只有可行而無熱情，會做得索然無味、成果平庸。



這是一張說明圖，點的位置是示意而非量測——因為「熱情」只有你自己能給分，而「可行性」要等你做完下一張圖的四本帳才知道。圖要說的是 18.3.1 節那句話的形狀：只有熱情而不可行（左上），會做到一半崩潰;只有可行而無熱情（右下），會做得索然無味。兩個反例值得記住。「自駕車完整系統」不是不好的題目，它只是不該是一學期四個人的題目;而「重做一次課本範例」一定做得完，卻什麼也學不到——因為你不會為它多熬一個晚上。一個實用的動作是：把題目先往右推。先定義一個「一定做得出來」的最小可行版本（單一品種、固定距離、固定打光），讓題目從左上角移進右上角，行有餘力再把範圍加回去。這比換一個你沒感覺的題目好得多。

圖 18-4　選題：熱情與可行性的交集

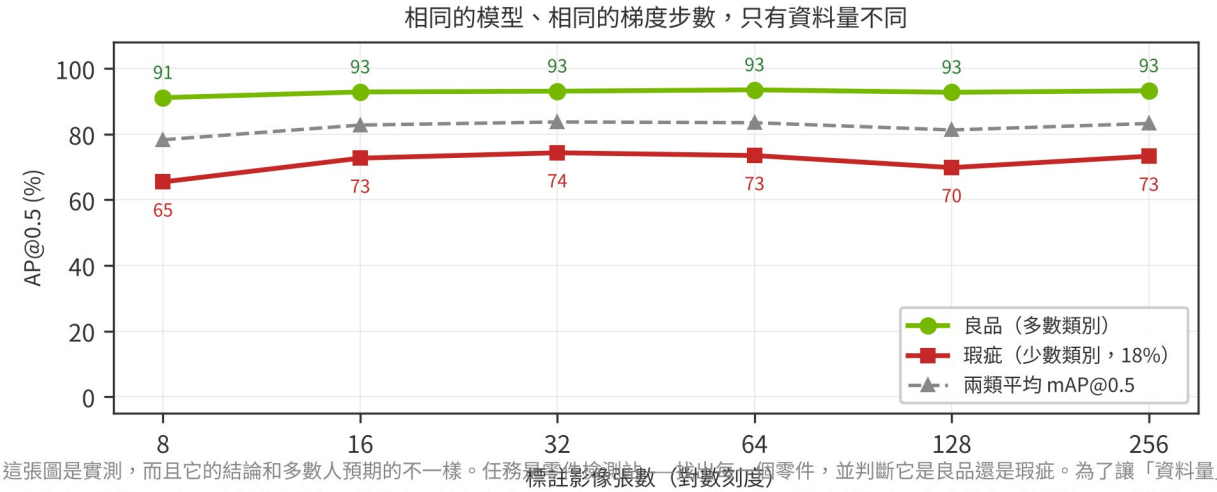
可行性的評估要誠實面對四件事：時間(一學期實際能投入多少小時?)、資料(需要多少標註影像?蒐集得到嗎?)、硬體(有沒有相機、Jetson、機構?)、技術落差(有多少是你沒學過、需要現學的?)。一個實用的技巧是設定「最小可行版本」：先定義一個「一定做得出來」的核心功能，確保專題有底線;行有餘力再往上加。這比訂一個宏大目標卻做不完要好得多。它的投資報酬率可以直接算。圖 18-9 在同一批現場影像上比了三條路線：照抄預設值的傳統管線零準備、寫完就能跑，平均數錯 3.20 個零件;同一條管線花約半天調過參數，誤差降到 0.28 個;而標註 256 張再訓練的偵測器，準備成本約四小時，誤差 1.20 個。也就是說，在「數有幾個」這件事上，半天的調參打敗了四小時的標註加訓練，而且快了約二十倍。要公平地補充：偵測器在這個實驗裡只拿到很小的訓練預算，而且它能做傳統管線完全做不到的事（判斷零件好壞，圖 18-13）。但這正是最小可行版本的價值所在——它幾乎免費，卻立刻告訴你三件事：這個任務的基準線在哪裡、它到底難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。至於可行性四件事裡的「資料」，也有一個比直覺可靠的估法：圖 18-5 量過本章這個任務每張影像平均有 14.1 個框要畫，以熟手每框四秒計算，標 256 張約四小時;若你的題目要框遮擋的果實或畫出瑕疵輪廓，一張十分鐘都有可能。這通常是專題裡最被低估的一筆帳。



18.3.1 節說可行性要誠實面對四件事。這張圖把前三件量出來，用的就是本章那個零件檢測站。帳一是標註：本任務每張影像平均有 14 個零件要框，以熟手每框 4 秒估算，標 128 張要 2.0 小時、256 張要 4.0 小時。聽起來不多，但請注意這是「一個人、一種零件、框很好畫」的情況;換成要框遮擋的果實或分割的瑕疵區域，一張十分鐘都有可能。這通常是專題裡最被低估的一筆帳。帳二是訓練：在本機純 CPU 上，每一次訓練約 5 到 7 分鐘，而你不會只訓練一次——調一次參數就要重跑一次。帳三是硬體：傳統管線 1.9 毫秒，YOLO26n 在 320 解析度下 22 毫秒，差了約 12 倍;這個數字決定你要不要 Jetson、要不要降解析度（第 17 章）。第四本帳「技術落差」沒有畫在圖上，因為它因人而異——但它有一個很好的量法：把你的系統拆成模組，逐一問「這個我寫得出來嗎」，答案是否的那幾個，就是你要在 M2 之前先試掉的風險。

圖 18-5　可行性的四本帳：把「做得完嗎」換算成數字

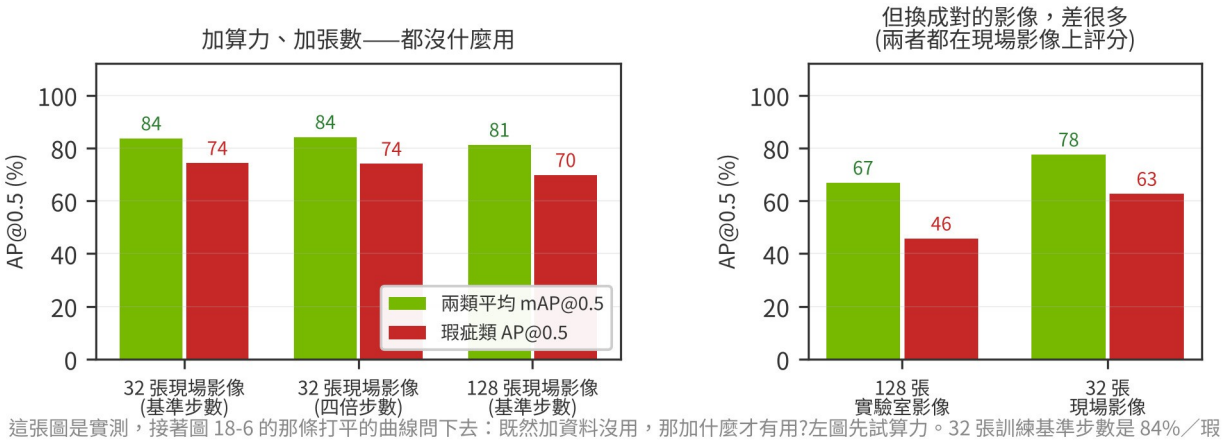
那麼資料到底要多少才夠?這個問題有一個比直覺可靠的答案，而且答案和多數人想的不一樣。圖 18-6 在本章這個零件檢測站上做了一條學習曲線：相同的模型、相同的梯度步數，只有標註張數不同。結果是「找得到零件」這件事八張就有九成以上，而「判斷它是良品還是瑕疵」始終落後二十個百分點上下——難的從來不是定位而是分類，這也解釋了為什麼很多專題「框抓得很準、就是判錯」。更值得注意的是曲線的形狀：瑕疵那一類從八張爬到三十二張之後就停了，三十二、六十四、一百二十八、兩百五十六張的成績彼此分不出高下。所以「我要標幾張」這個問題，問法本身就有點問題。



這張圖是實測，而且它的結論和多數人預期的不一樣。任務是零件檢測站，找出每一個零件，並判斷它是良品還是瑕疵。為了讓「資料量」成為唯一的變因，每一次訓練都給相同的梯度更新步數（而不是相同的 epoch 數），否則小資料集會因為步數少而被低估，量到的就不是資料的價值而是算力的價值。驗證集固定 60 張，與任何訓練集都不重疊。結果有三件事。第一，兩條線的高度差很大而且一路不收斂：「找得到零件」很容易，8 張就有 91%;「判斷它壞了沒有」始終落後二十個百分點上下。難的從來不是定位而是分類——這解釋了為什麼很多專題「框抓得很準、就是判錯」。第二，進步集中在最前面。瑕疵那一類從 8 張的 65% 升到 32 張的 74%，之後就停了：64、128、256 張分別是 73%、70%、73%，彼此的高低完全在量測噪聲裡（推導 18.5.3 會說明六十張驗證集的標準誤就有三到四個百分點）。也就是說，第三十三張到第二百五十六張，幾乎什麼也沒買到。第三——這才是重點——打平不代表「資料夠了」，而代表「這批資料的變異用完了」。本實驗的影像都由同一個生成器產出：同一種零件、同一個尺寸範圍、同樣那幾種瑕疵。第三十三張與第三張長得太像，所以模型學不到新東西。真實專題裡一模一樣：在同一張桌子、同一個下午、同一個角度拍兩百張，遠不如在十個不同的時間與位置各拍二十張。下一張圖會直接把這件事量出來。所以排標註預算時要問的不是「幾張」，而是「我涵蓋了多少種會發生的情況、每一種有幾個少數類別的樣本」。

圖 18-6　要標多少張才夠：學習曲線

打平不代表資料夠了，而代表這批資料的變異用完了。圖 18-7 把這件事直接量出來：同一批三十二張影像多跑四倍的訓練步數，成績幾乎不動;張數加到一百二十八張，成績也幾乎不動;但若把訓練影像換成與使用現場相同條件的影像，mAP 就從 66.9% 升到 77.6%，瑕疵類更高出十七個百分點——而那只用了四分之一的張數。道理很單純：模型學得到的只有你給它看過的變異，在實驗室拍一千張，它就只學會實驗室。所以可行性評估裡的「資料」那一欄，該寫的不是「我能拍幾張照片」，而是「我涵蓋得到幾種光線、角度與距離，每一種少數類別各有幾個樣本」。這也直接決定了排程：資料蒐集要在 M2 就去現場做一次，不要等到 M4 整合時才發現訓練集和現場是兩回事。



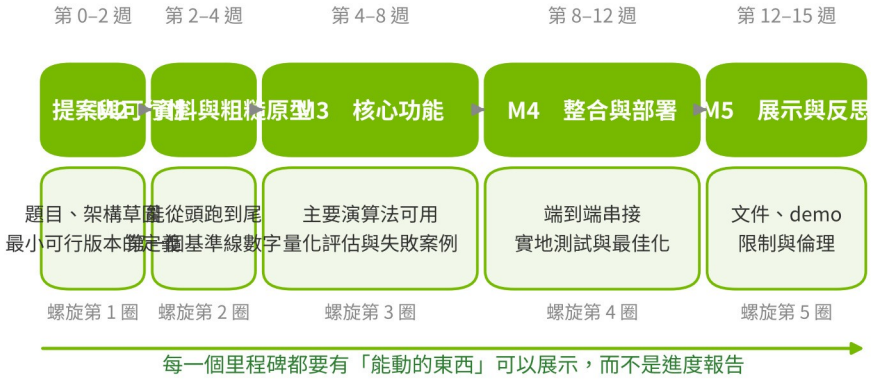
這張圖是實測，接著圖 18-6 的那條打平的曲線問下去：既然加資料沒用，那加什麼才有用?左圖先試算力。32 張訓練基準步數是 84%／瑕疵 74%;同一批 32 張跑四倍的梯度步數，只動到 84%／74%;把張數加到 128 張是 81%／70%。三根柱子幾乎一樣高——算力與張數在這裡都已經不是瓶頸了。右圖換一個變因。兩個模型都在「現場」那種打光不穩的影像上評分，但其中一個是用 128 張理想取像條件下的影像訓練的（也就是在實驗室裡拍得漂漂亮亮的那種），另一個只用 32 張真正的現場影像。結果是 67% 對 78%，相差 11 個百分點;瑕疵那一類相差 17 個百分點。四分之一的張數，贏了四倍的張數。這就是整張圖的結論，也是 18.3.1 節「在真實環境蒐集訓練資料」那句話的量化版本：模型學得到的只有你給它看過的變異。在實驗室拍一千張，它就只學會實驗室;而現場的斜向打光、暗角、刮痕與失焦，它一次也沒見過。對專題的建議因此很具體：與其在同一個下午多拍兩百張，不如分成十個不同的時間、位置與光線各拍二十張;而且資料蒐集這件事，應該在 M2 就去現場做一次，不要等到 M4 才發現訓練集和現場根本是兩回事。

圖 18-7　打平之後，該加算力還是加變異

選題方向的建議：從你熟悉的生活場域出發——雲林在地的農業(果實計數、病害辨識、雜草偵測)、製造業(工件檢測、儀表判讀、安全監控)、校園生活(教室人數統計、器材管理、無障礙輔助)、或個人興趣(運動姿勢分析、寵物監控、遊戲互動)。在地與生活化的題目，往往最能激發熱情，也最容易取得資料。

18.3.2 里程碑規劃

把一學期的專題切成四到五個里程碑，每個里程碑對應創意學習螺旋的一圈——都要有可展示的成果，而非只是「進度報告」：

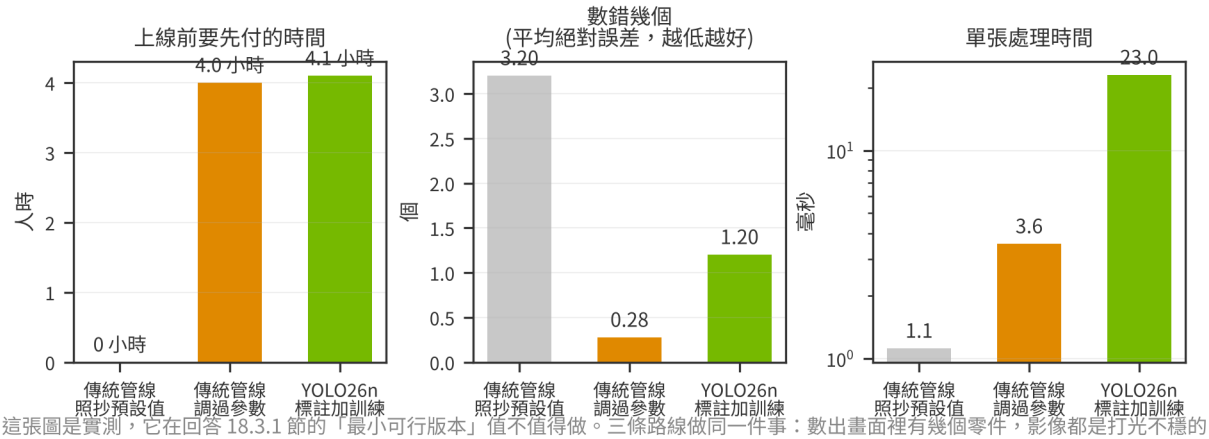


這是一張規劃用的說明圖，週數可依你的學期調整。它要強調的只有一件事：每一格的下半部寫的是「交付物」，而不是「做了什麼」。「這兩週我讀了很多論文」不是里程碑，「這兩週我做出一個會動但很爛的版本，準確率 31%」才是。理由在 18.3.2 節說過，而圖 18-3 剛剛量過：改進來自上一輪量到的結果，所以你必須儘早有一個可以量的東西。M2 是整個時程裡最關鍵的一格——它的任務不是做得好，而是「證明這條路走得通」。若 M2 做不出來，你還有十三週可以換方向;若拖到 M4 才發現，就沒有退路了。

圖 18-8　里程碑：一學期要繞幾圈

表 18-1 專題里程碑規劃

請特別重視 M2 的「粗糙原型」：許多專題失敗的原因，是把太多時間花在規劃與準備，直到後期才發現核心技術根本行不通。先做一個能動的版本(哪怕只有百分之三十的準確率)，你就能及早發現真正的困難點，並據此調整方向——這正是遊玩精神的工程版本。



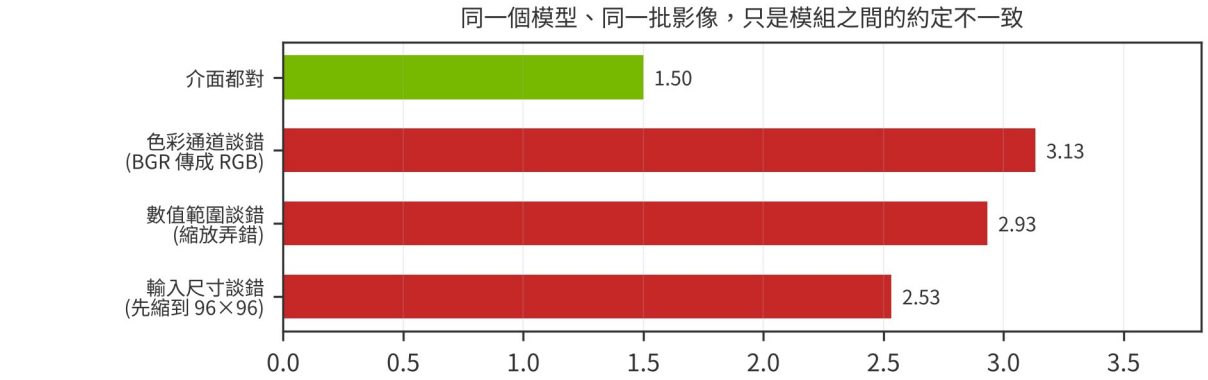
這張圖是實測，它在回答 18.3.1 節的「最小可行版本」值不值得做。三條路線做同一件事：數出畫面裡有幾個零件，影像都是打光不穩的那種。灰色是照抄範例預設值的傳統管線——零準備、寫完就能跑，但平均數錯 3.2 個。橘色是同一條管線，花大約半天把參數調過一輪（圖 18-3 的那五輪），平均誤差降到 0.28 個。綠色是標了 256 張、訓練過的偵測器，準備成本約 4.1 小時，平均誤差 1.20 個。這個結果值得停下來想一想：在「數有幾個」這件事上，花半天調參數的傳統管線，比花四小時標註加訓練的偵測器還準，而且快了 6 倍。要公平地說明為什麼：偵測器在這裡只給了很小的訓練預算（圖 18-6 的相同步數），而且它的工作點（信心門檻）也只是在開發集上粗調過;給它更多資源它會更好。但這恰恰是 M2 的論點——在你還沒有資源的那兩週，最小可行版本讓你立刻知道這個任務的基準線在哪裡、難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。

圖 18-9　最小可行版本有多便宜

18.3.3 團隊協作與工程實務

三個實務建議，它們是專業工程師的基本功：

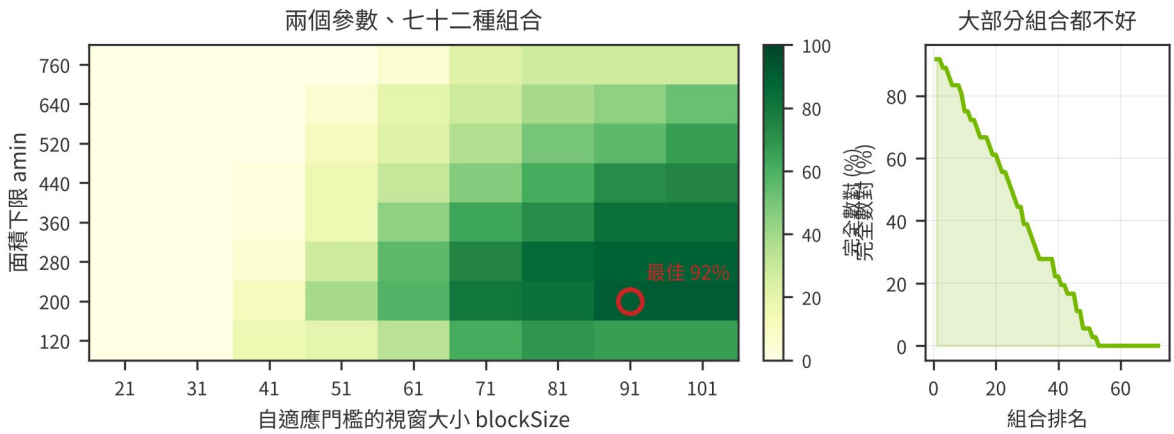
- 版本管理：用 Git 管理程式碼與文件。它不只是備份，更讓你能安心嘗試(改壞了隨時可以回復)、讓多人能平行開發、也讓你能回頭檢視「當時為什麼這樣改」。
- 分工與整合：依模組分工(如影像處理、模型訓練、硬體整合、介面)，但務必約定好模組之間的介面(輸入什麼、輸出什麼格式)，並儘早進行整合測試。太晚整合是團隊專題最常見的災難。



這張圖是實測，量的是 18.3.3 節說的「太晚整合是團隊專題最常踩的坑」那一種——三個介面都對，但整合時有一個程式是壞的——模型沒問題、影像沒問題，壞掉的只是模組之間的約定。介面都對時，平均數錯 1.50 個;把 BGR 當成 RGB 傳進去（OpenCV 與多數深度學習框架的順序相反，這是最經典的一個）變成 3.13 個;前處理時把縮放談錯是 2.93 個;負責擷取的人先把影像縮到 96×96 再交出來，是 2.53 個。還有一種看不見的介面錯誤沒有畫在圖上：偵測模組輸出 xyxy、下游卻按 xywh 解讀。這時數量完全正確，但框的平均 IoU 只剩 0.04——畫出來的框會整齊地偏向右下角，而系統不會報錯，它只會安靜地算錯位置。這類錯誤的共同特徵，正是它們難抓的原因：每一個模組單獨測試都是對的，合起來才錯;而且不會丟出例外，只會讓數字變難看，於是團隊往往花好幾天在調模型，卻從沒懷疑過介面。對策也很單純：在 M1 就把每個模組的輸入輸出寫成白紙黑字（格式、順序、數值範圍、座標系、單位），並在 M2 就跑一次端到端——哪怕每個模組裡面都還是假的。

圖 18-10　整合：三個介面不一致的真實代價

- 工程筆記：記錄每次實驗的參數、結果與觀察。三個月後你絕對不會記得「那次準確率 92% 是用什麼設定跑出來的」。這份筆記也是期末報告最寶貴的素材。這兩條建議都可以量，而且量出來的數字比建議本身更有說服力。先看整合。圖 18-10 做了一個實驗：模型沒換、影像沒換、程式也沒有任何一行是壞的，只是把模組之間的約定弄錯。介面都對時平均數錯 1.50 個零件;把 BGR 當成 RGB 傳進去（OpenCV 與多數深度學習框架的通道順序相反，這是最經典的一個）變成 3.13 個;數值範圍談錯是 2.93 個;負責擷取的人先把影像縮到 96×96 再交出來是 2.53 個。還有一種更陰險的：偵測模組輸出 xyxy、下游卻按 xywh 解讀——數量完全正確，框的平均 IoU 卻只剩 0.04，畫出來會整齊地偏向右下角，而系統不會報任何錯。這類錯誤難抓的原因正是每個模組單獨測都對、合起來才錯，而且症狀只是「數字有點難看」。再看筆記。圖 18-11 把兩個參數交叉掃過 72 種組合：最好的一格是 91.7%，中位數只有 27.8%，而落在最佳值五個百分點以內的只有 4 格。參數空間的好區域就是這麼小，所以「我隨便試了幾個值」等於沒調過;而三個月後要回答那個 91.7% 是怎麼跑出來的，只有紀錄救得了你。順帶一提，那張圖還顯示參數之間會互相影響（面積下限可以調得很小，是因為前一步的開運算已經清掉雜訊了），所以筆記要記的是整組設定，而不是單一數字。



這張圖是實測，也是 18.3.3 節「寫工程筆記」那一條的理由。左圖把兩個參數（自適應門檻的視窗大小、連通元件的面積下限）交叉掃過 72 種組合，每一格都真的跑過 36 張影像。最好的一格是 92%，而 72 格裡只有 4 格落在最佳值的五個百分點之內——右圖把同樣的資料排序，可以看到絕大多數的組合都不好。請把這張圖當成一個地形圖：你在專題裡做的每一次「調調看」，都是在這個地形上踩一個點。兩件事因此變得很實際。第一，你不可能記得住。三個月後要回答「那次 92% 是怎麼跑出來的」，只有筆記救得了你——而且期末報告正需要這張表。第二，地形有結構，而且結構會說話。好的區域集中在「視窗夠大、面積下限不要太高」那一角：視窗太小時自適應門檻會把零件內部也切開，面積下限拉太高則開始把真正的零件濾掉。值得注意的是面積下限調得很小並沒有被懲罰——因為前一步的開運算已經把雜訊點清掉了。這就是參數之間的交互作用：單獨調一個參數得到的結論，換一個前處理就可能不成立，所以筆記要記的是「整組設定」，而不是單一數字。順帶一提，這整張圖跑完只要幾十秒——傳統管線最大的優勢就在這裡：你可以把整個參數空間掃過一遍，而深度學習的每一格都要重訓一次（圖 18-7 量過那個代價）。

圖 18-11　工程筆記：參數空間長什麼樣子

18.3.4 推導：一學期到底有多少工時，以及它會被什麼吃掉

18.3.1 節要你誠實評估時間，但沒說怎麼估。這一小節把它算出來——因為多數專題不是敗在技術，而是敗在一開始就把時間算錯了。

先算總額。一個四人小組，一學期十五週，每人每週實際投入三小時（這個數字請自己誠實填，它要扣掉你其他五門課、打工與社團）：

```
總工時 = 人數 × 週數 × 每人每週時數
       = 4 × 15 × 3 = 180 人時
```

一百八十人時聽起來很多。但它會先被三件事吃掉，而且是必然的。第一是協作本身：開會、同步進度、解釋自己做了什麼、合併彼此的程式碼——這在四人小組裡通常要佔掉一到兩成，而且人愈多佔比愈高。第二是環境與工具：裝套件、版本衝突、某個人的電腦跑不動、相機驅動不認。第三是文件與展示，也就是 M5 那一格。保守地各抓 15%、10%、15%，剩下的是：

```
可用於開發的工時 = 180 × (1 − 0.15 − 0.10 − 0.15)
                 = 180 × 0.60 = 108 人時
```

接著把本章量到的數字填進去。圖 18-5 量過：這個零件檢測站每張影像平均有十四個框要畫，以熟手每框四秒估算，標註兩百五十六張約四小時;而你的題目若要框遮擋的果實或畫出瑕疵的輪廓，一張十分鐘都有可能。同一張圖也量過訓練：在筆電 CPU 上每訓練一次要五到七分鐘，而你不會只訓練一次——圖 18-11 那張參數地形圖提醒你，每一次「調調看」都是地形上的一個點，傳統管線掃七十二個點只要幾十秒，深度學習掃七十二個點要重訓七十二次。

把這些擺在一起，一百零八人時的分配大概長這樣：標註與資料整理十五人時、去現場重新蒐集一輪十人時（圖 18-7 說明了為什麼這一筆不能省）、核心開發與調參

四十人時、系統整合二十人時（圖 18-10 說明了為什麼不能少）、實地測試與退化量測十人時（圖 18-19 的那六格）。加起來已經滿了——而且這裡面沒有任何一格是留給「重做」的。

這就帶出本小節真正的結論，也是 18.3.1 節那個「最小可行版本」的算術理由。既然預算裡沒有重做的空間，那麼唯一能降低風險的做法，就是讓「發現做不到」這件事儘早發生。若核心技術行不通，在第四週發現，你還有十一週可以換方向;在第十二週發現，你只能交一份失敗的報告。把這句話翻成排程的語言：最不確定的那一段要排在最前面，而不是最後面。這與一般人的直覺相反——大家習慣先做有把握的部分（介面、簡報、架構圖），因為那會讓人覺得有進度;但有進度跟降低風險是兩件事。

最後給一個很實際的檢查法，可以在 M1 就用：把你的系統拆成模組，為每一個模組標上「我有多確定做得出來」。只要有任何一個標著「不確定」，M2 的任務就是把那一個先試掉——不是把它做完，而是做一個粗糙到不行、但足以回答「行不行」的版本。這正是圖 18-9 的意義：傳統管線的那條路線準備成本是零，所以它幾乎永遠值得在第二週先跑一遍，哪怕你最後不會用它。它給你的不只是一個基準線，更是一個「這個任務到底難在哪裡」的答案。

18.4 專題方向與技術地圖

本節把全書十七章的技術整理成一張「技術地圖」，幫助你在選定題目後，知道該回去複習哪幾章。

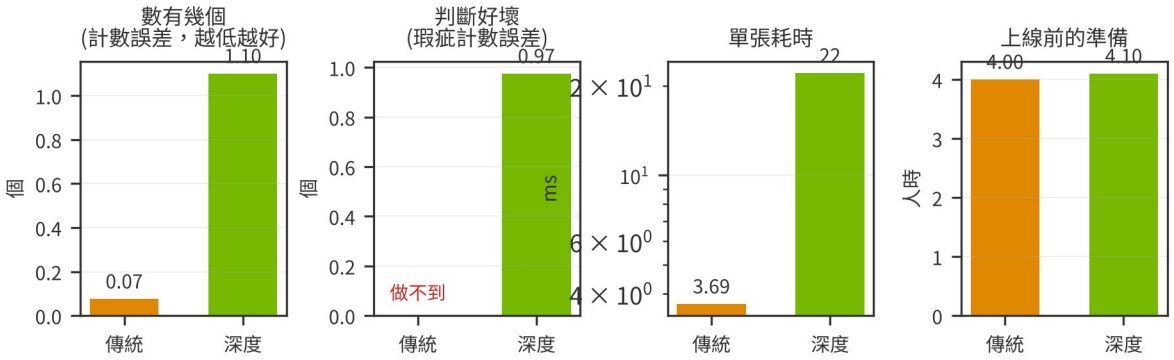


這是一張索引，不是實測。18.4 節說「不要為了用最新的技術而選題，要為了解決問題而選技術」，這張表就是那句話的操作方式：先把你的系統拆成「它到底要回答什麼問題」，再沿著那一列回去找工具。請特別注意第一列與第二列的差別——它是本章用整個零件檢測站量出來的一件事：「數有幾個」很快就能做到堪用（圖 18-6 顯示八張標註就有七成以上），「判斷好壞」卻要多得多的資料。所以當你的題目同時需要這兩件事時，請把它們當成兩個里程碑，而不是一個。另外，每一列的第二欄都同時列了傳統與深度學習的路線，這是刻意的：圖 18-13 會把兩者擺在一起量，而結論不是誰比較好，是它們貴在不同的地方。

圖 18-12　技術地圖：你的題目要回去複習哪幾章

表 18-2 專題方向與技術地圖

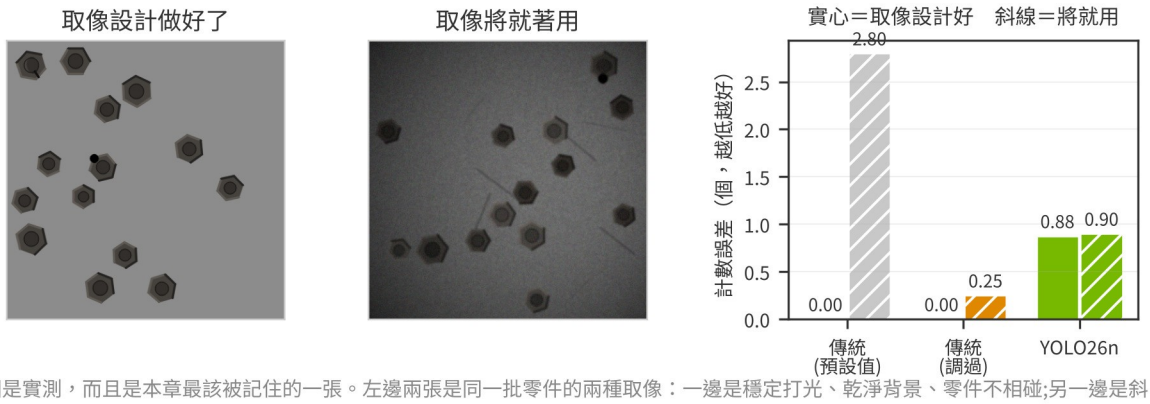
選題時的一個重要提醒：不要為了用最新的技術而選題，要為了解決問題而選技術。承第 9、10、13、16 章反覆強調的判準——一個用 Otsu 加連通元件就能穩定運作的零件計數系統，價值不亞於一個用上最新模型卻不穩定的系統。工程的價值在於解決問題，不在於炫技。這句話本書說過很多次，這裡用一個完整的實測把它釘下來。圖 18-13 讓兩條路線做同一件事，在同一批現場影像上評分。計數上傳統管線平均錯 0.07 個、偵測器錯 1.10 個——傳統管線贏;速度上是 3.7 毫秒對 22 毫秒;準備成本是半天對四小時。但第二格是分水嶺：判斷零件好壞，傳統管線做不到。它只會告訴你「這裡有一塊連通的東西」，不會告訴你那塊東西上有沒有裂紋或缺角。所以結論不是誰比較好，而是它們貴在不同的地方，而選擇的邏輯很清楚：問題若只是「有幾個」而且場景固定，傳統管線又快又準又便宜;一旦要回答「它長什麼樣子」，那筆標註的錢就非花不可。



這張圖是實測，四根柱子回答四個不同的問題——而這正是 18.4 節「適合勝於最強」的具體樣子。兩邊都在打光不穩的現場影像上評分，傳統管線用的是圖 18-3 調出來的參數，偵測器用的是圖 18-6 訓練出來的模型與開發集挑出的信心門檻。第一格是計數：傳統管線平均數錯 0.07 個，偵測器 1.10 個。傳統管線贏——這不是筆誤，而是因為它被針對這個場景調過，而偵測器只拿到很小的訓練預算。第二格才是分水嶺：傳統管線做不到。它只會告訴你「這裡有一塊連通的東西」，不會告訴你那塊東西上有沒有裂紋或缺角;偵測器的瑕疵計數平均誤差是 0.97 個。要用傳統方法做外觀判斷不是不行，但得再接上第 6 章的紋理分析或第 13、14 章的特徵加分類器，也就是再長出一整段管線。第三格是速度：3.7 毫秒對 22 毫秒。第四格是準備成本：半天對 4.1 小時。把四格一起看，選擇的邏輯就清楚了：問題若只是「有幾個」而且場景固定，傳統管線又快又準又便宜;一旦要回答「它長什麼樣子」，那筆標註的錢就非花不可。真正的錯誤不是選錯，而是沒有量過就選——包括反過來的那種錯：因為深度學習比較潮，就用它去做一件 Otsu 就能解決的事。

圖 18-13　兩條路線的四面比較

還有一個比選模型更早、槓桿也更大的決定。圖 18-14 量了同一批零件的兩種取像。取像設計做好時（穩定打光、乾淨背景、零件不相碰），連照抄預設值的傳統管線都是零誤差;取像將就著用，同一段程式就錯到 2.80 個。要把它救回來，你可以花半天調參數（降到 0.25 個）或花幾小時標註與訓練（0.90 個）——兩者都有效，但都比不上一開始就把光打好，而那幾乎不花錢。這正是第 7、8、17 章一再說的：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然也要誠實：果園、戶外、移動中的機器人根本沒有「把光控制好」這個選項，那時標註的錢就是你必須付的。所以專題的第一個決定不是選模型，而是問：我的現場，屬於哪一種?



這張圖是實測，而且是本章最該被記住的一張。左邊兩張是同一批零件的兩種取像：一邊是穩定打光、乾淨背景、零件不相碰;另一邊是斜向打光造成的明暗不均、鏡頭暗角、背景刮痕、零件亮度不一、輕微失焦與雜訊——也就是「相機隨便架一架」的樣子。右圖量的是計數誤差（越低越好），三組、各兩種取像條件。先看最左邊那一組：取像設計做好時，連照抄預設值的傳統管線都只錯 0.00 個;取像將就著用，同一段程式錯到 2.8 個。一盞穩定的光源與一塊素色背板，就是這兩根柱子之間的差距——而它幾乎不花錢。另外兩組是「取像救不回來時」你能做的兩件事：花半天調參數（中間一組，把誤差拉回 0.25 個），或花幾小時標註與訓練（右邊一組，0.90 個）。兩者都有效，但都比不上一開始就把光打好。這正是第 7、8、17 章一再說的：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然也要誠實：果園、戶外、移動中的機器人根本沒有「把光控制好」這個選項，那時右邊那兩組柱子就是你必須付的錢。所以專題的第一個決定不是選模型，而是問：我的現場，到底屬於哪一種?

圖 18-14　把錢花在打光上、調參上，還是標註上

18.5 成果展示與評估

18.5.1 如何做一場好的展示

技術做得好，還要說得清楚——這是工程師常被低估的能力。一場好的專題展示建議這樣安排：



這是一張建議，不是實測;時間比例請依你的場合調整。它唯一想對抗的是一種很常見的報告結構：花八分鐘講「我用了什麼模型、調了什麼參數」，最後三十秒才放一段 demo。18.5.1 節的順序反過來——先讓聽眾知道你在解決什麼問題，再儘早讓他們親眼看到東西會動，然後才解釋你怎麼做到的。第四格「誠實呈現限制」被標成重點色，是因為它最反直覺。學生常以為講出系統的弱點會扣分，但一個說得出「在逆光時召回率會從九成掉到六成、所以我加了一道亮度檢查」的人，顯然比一個只報最好那個數字的人更理解自己的系統。而且評審一定會問——與其被問倒，不如自己先講。

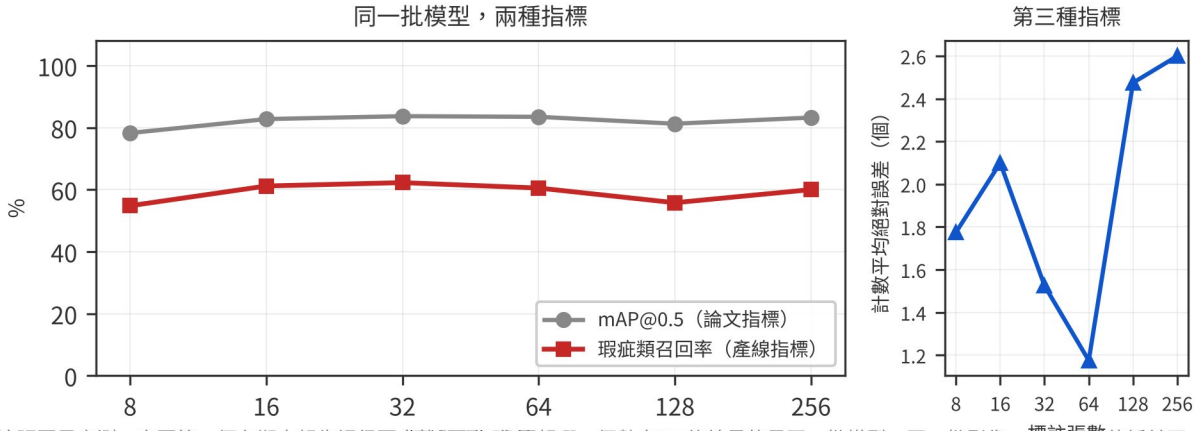
圖 18-15　一場好的展示：順序與時間

1. 從問題出發：先說「為什麼要做這個」，用一個具體的情境或痛點抓住聽眾。不要一開場就講你用了什麼模型。
2. 展示成果：盡早讓聽眾看到「它會動」——現場 demo 或實測影片，比任何投影片都有說服力。

3. 說明方法：再回頭解釋你怎麼做到的，重點放在關鍵決策與取捨(為什麼選這個方法而非那個)，而非流水帳。
4. 誠實呈現限制：說明系統在什麼情況下會失敗、目前的準確率、還有什麼沒做到。誠實不會扣分，反而顯示你真正理解自己的系統——這也呼應第 7、16 章的工程倫理。
5. 反思與展望：分享你學到什麼、若重來會怎麼做、下一步可以往哪走。

18.5.2 評量向度

本專題的評量對應 4P 精神，不只看最終成果，更看過程與成長。在看向度表之前，先講一件常被忽略的事：你要怎麼報你的數字。圖 18-16 拿同一批模型量了三種指標，結果彼此矛盾。mAP@0.5 一路在 78% 到 84% 之間，相當好看;但「實際有瑕疵的零件裡抓出來幾成」只有 55% 到 62%，始終低二十個百分點以上;而計數誤差又是另一條曲線，甚至不是單調的。也就是說，一個 mAP 看起來八成的系統，可能正讓四成的不良品流到客戶手上。三個指標、三種結論，所以驗收指標必須在 M1 就寫清楚，而且要從使用者的痛點反推，不是從你會算的東西挑。另外兩個會讓數字偏離真相的陷阱也要先說。一是類別不平衡：圖 18-18 的訓練集有 2842 個良品、只有 643 個瑕疵，結果良品 AP 93.2%、瑕疵 73.3%，而兩類平均 83.2% 離良品很近、離瑕疵很遠;召回率的差距更大，99.6% 對 60.0%。平均值會把你最在乎的那一類藏起來，所以永遠要報逐類別的數字。二是資料外洩：圖 18-17 故意讓二十張驗證影像也進訓練集，mAP 量到 88.3%;同一個模型在真正沒看過的影像上只有 81.0%，瑕疵類更從 80.2% 掉到 69.9%。那些百分點憑空出現，而且不會發出任何警告。以下是評量向度



這張圖是實測，它回答一個在期末報告裡很要命的問題：標註影像張數報哪一個數字?三條線量的是同一批模型、同一批影像，標註張數的話並不一樣。灰線是 mAP@0.5，也就是論文與比賽用的指標，從 78% 爬到 83%，一路都很好看。紅線是產線真正在乎的那個數字——實際有瑕疵的零件裡，有多少被抓出來——從 55% 到 60%，始終比 mAP 低二十個百分點以上。換句話說，一個 mAP 看起來八成的系統，可能正讓四成的不良品流到客戶手上。右邊那條藍線又是另一回事：若你的客戶只要「這一盤有幾個零件」，那麼他要看的是計數誤差，而它在 8 張時是 1.8 個、到 256 張是 2.6 個——這條線甚至不是單調的。三個指標、三種結論，而且它們的高低順序會隨資料量變來變去。所以 M1 的提案裡就應該寫清楚「驗收指標是什麼」——而且那個指標要從使用者的痛點反推，而不是從你會算的東西挑。一個漏掉四成瑕疵的系統，mAP 可能有八成;若你在報告裡只報那個八成，不算說謊，但也不算誠實。

圖 18-16　指標選錯，你會看到完全不同的世界

表 18-3 專題評量向度

請注意最後一列「反思與倫理」。承第 7 章的「還原 vs. 腦補」、第 9 章的醫學影像、第 12 章的照護監控、第 14 章的資料偏誤、第 16 章的 Redmon 故事、第 17 章

的隱私設計——在你的專題報告中，請務必回答三個問題：誰會從這個系統受益?誰可能受到傷害?如果它出錯了，後果由誰承擔?這不是加分題，而是專業工作者的基本功。

18.5.3 推導：你測了幾張，那個準確率才算數

期末報告裡最常見的一句話是：「我們測了二十張，全部正確，準確率 100%。」這句話的問題不在於測試，而在於它把一個很小的樣本說成了一個確定的事實。這一小節把「那個數字可以相信到什麼程度」算出來，它承接第 14.5.3 節的標準誤，而且只需要一條式子。

你量到的準確率是一個比例。假設系統真正的成功率是 p，你測了 n 次，那麼量到的比例 p̂ 會在 p 附近擺盪，擺盪的幅度是：

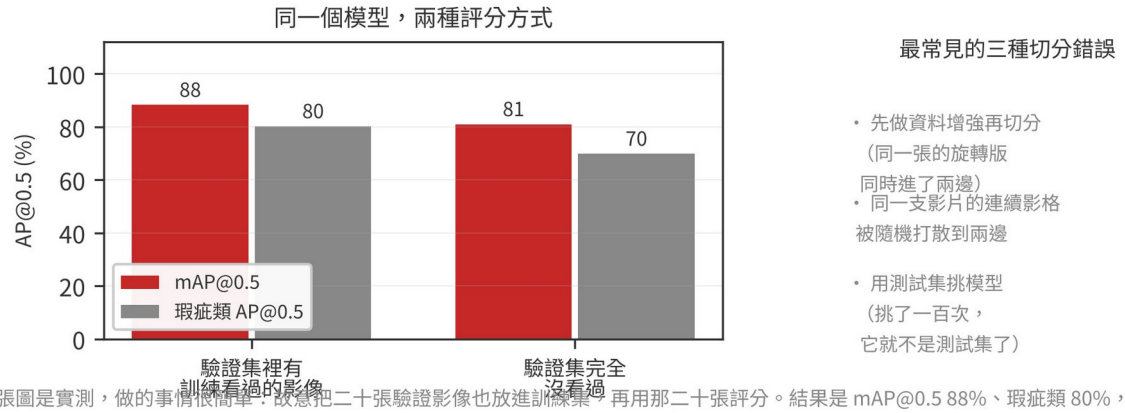
```
標準誤 SE = √( p(1−p) / n )
約 95% 信賴區間 ≈ p̂ ± 1.96 × SE
```

代幾個數字進去，結論會讓人清醒。測 20 張、量到 90%：SE = √(0.9×0.1/20) = 0.067，區間約是 77% 到 100%。也就是說，你報的「九成」和「八成」在統計上分不出來。測 100 張、量到 90%：SE = 0.03，區間約 84% 到 96%。測 500 張：SE = 0.013，區間約 87% 到 93%。要把區間縮小一半，樣本數要變四倍——因為 n 在根號裡面。這條平方根關係值得記住，它決定了你該測多少、也決定了測到某個程度就該停。

那「二十張全對」呢?這個情況特別，因為 p̂ = 1 時上面那條式子給出 SE = 0，顯然不對。統計上有一個好用的近似叫三倍法則（rule of three）：若 n 次測試都沒有失敗，那麼真正的失敗率上限約是 3/n。

```
n 次全對 → 失敗率的 95% 上限 ≈ 3 / n
  n = 20  → 15%
  n = 100 → 3%
  n = 300 → 1%
```

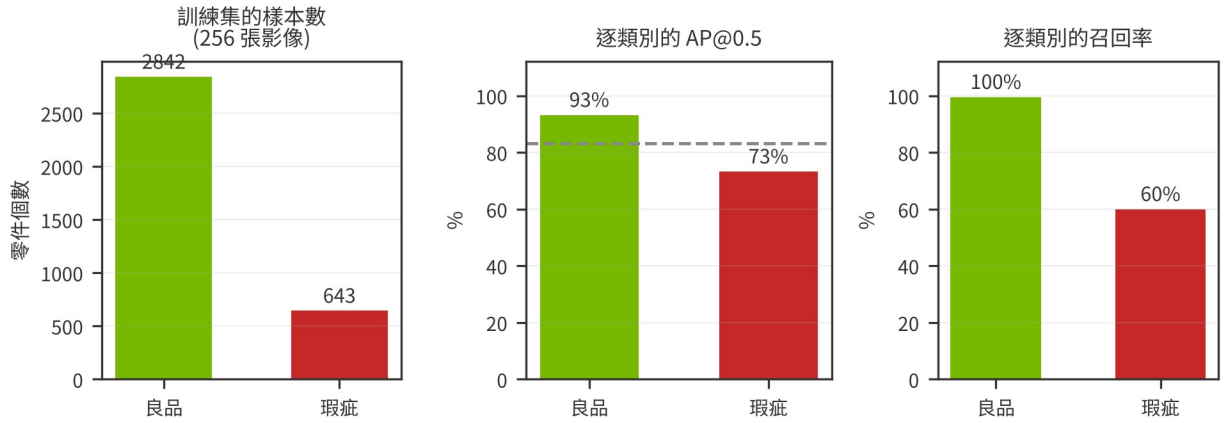
所以「二十張全對」真正的意思是：「失敗率有九成五的把握低於 15%」——這和 100% 差得很遠。若你的系統要宣稱不良率低於 1%，光是驗收就得測三百張以上，而且那三百張必須是模型沒看過的（圖 18-17）。



這張圖是實測，做的事情很簡單：故意把二十張驗證影像也放進訓練集，再用那二十張評分。結果是 mAP@0.5 88%、瑕疵類 80%，漂亮得不得了。接著拿同一個模型去評真正沒看過的影像：81% 與 70%，分別掉了 7 與 10 個百分點。這 7 個百分點是憑空出現的，而且它不會發出任何警告——程式不會報錯、損失曲線很漂亮、驗證分數還一路上升。第 14 章與第 16 章都提過這件事，這裡再提一次是因為它在專題裡特別容易發生：右邊列的三種情況都不是粗心，而是看起來完全合理的操作。尤其是第三種——你每次「用測試集看看這版好不好」然後決定留哪一版，就在把測試集的資訊搬進模型裡，這和圖 18-3 裡開發集與測試集愈拉愈開是同一個機制。專題的做法：在 M2 就把資料切成三份並寫進文件，測試集鎖起來，整學期只在 M5 驗收時開一次。

圖 18-17　切分錯了，你會量到一個不存在的好成績

還有兩件事會讓情況比式子說的更糟，必須一起講。第一，上面的算式假設每次測試互相獨立。若你的三百張影像是同一段影片的連續影格，它們彼此高度相似，等效的樣本數遠小於三百——這也是圖 18-17 右邊列出的切分錯誤之一。第二，少數類別的樣本數才是真正的瓶頸。圖 18-18 量過：訓練集裡良品有兩千八百多個、瑕疵只有六百多個，所以「瑕疵召回率」那個數字的區間，比整體準確率寬得多。要報一個可信的瑕疵召回率，你需要的是「夠多的瑕疵」，而不是「夠多的影像」。



這張圖是實測，追的是圖 18-6 留下的那條線索。左圖是訓練集真正的樣子：256 張影像裡有 2842 個良品、只有 643 個瑕疵，比例約 18%——這不是設計失誤，真實產線的不良率本來就低，這也正是瑕疵檢測難的根本原因：你最想抓的那一類，樣本最少。中圖是後果：良品 AP 93%、瑕疵 AP 73%，而兩類平均是 83%（灰色虛線）。注意平均值離良品很近、離瑕疵很遠——因為它是「兩類取平均」，而若你改用「所有樣本一起算」的指標（例如整體準確率），少數類別會被淹得更徹底。右圖是召回率，也就是「實際的瑕疵裡抓到幾成」：60%。對一條產線來說，這個數字才是真正要簽字負責的那個。專題上的三個實作建議：一、永遠報逐類別的數字，不要只報平均;二、排標註預算時針對少數類別排（要的是「兩百個瑕疵樣本」，不是「兩百張影像」）;三、若少數類別實在收集不到，考慮換個問法——用第 14 章的異常偵測思路「學會正常長什麼樣，不正常就報警」，往往比硬訓練一個樣本極少的類別實際。

圖 18-18　平均值藏起來的那一類

最後把它變成專題可以直接照做的三句話。一、報數字時一併報樣本數，寫「在 120 張未見影像上 mAP@0.5 為 86%」，而不是「準確率 86%」。二、小樣本就不要報到小數點後一位——那個精度是假的。三、若某一類的樣本數少於三十，誠實寫出來並說明這是目前的限制。這三句話不會讓你的成績變差，反而正是 18.5.2 節那一列「誠實呈現限制」要看的東西。

18.6 專題案例參考

以下四個案例示範不同類型的專題如何整合全書技術。它們不是標準答案，而是幫助你想像可能性。

18.6.1 案例一：果園產量估計系統

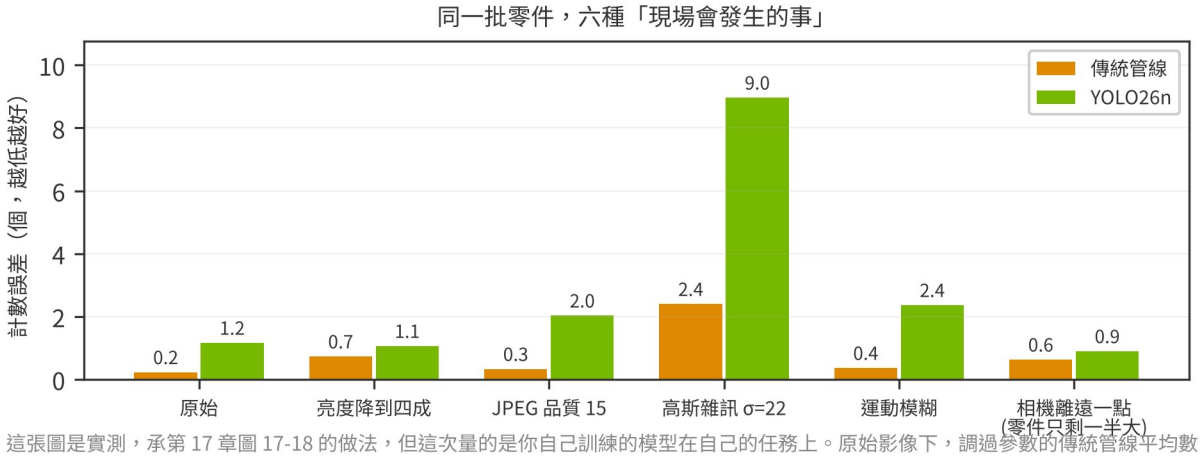
問題：小農難以在採收前準確估計產量，影響銷售規劃與人力調度。做法：用手機或樹莓派 5 配 AI Camera 拍攝果樹 → 以 YOLO 偵測並計數果實(第 16 章)→ 用第 8 章的色彩分析判斷成熟度 → 用第 4 章的相機校正估計果實大小 → 統計輸出估產報告。

技術挑戰與對策：果實相互遮擋(可結合多角度拍攝，或訓練模型學習遮擋樣態)、光線變化劇烈(第 5、8 章的前處理)、標註資料不足(第 15 章的遷移學習與資料增強)。最小可行版本：先做單一品種、固定距離拍攝的計數，再逐步擴充。

18.6.2 案例二：工件瑕疵檢測站

問題：人工目視檢測易疲勞、標準不一致。做法：固定打光與相機(第 17 章的取像設計)→ 前處理拉平背景(第 5 章 CLAHE 或第 10 章頂帽)→ 雙軌檢測：傳統管線(第 6 章頻率域紋理異常、第 9、10 章分割與型態學)與深度學習(第 16 章 YOLO)並行 → 比較兩者的準確率、速度與對「未見過瑕疵」的表現。

這個題目的教育價值在於：它讓你親身體驗全書反覆強調的判準——傳統方法不需瑕疵樣本、可解釋、運算輕;深度學習強健、能處理複雜形態。透過實測數據來論證「什麼時候該用什麼」，遠比背誦結論深刻。



這張圖是實測，承第 17 章圖 17-18 的做法，但這次量的是你自己訓練的模型在自己的任務上。原始影像下，調過參數的傳統管線平均數錯 0.23 個、偵測器 1.17 個。接著六種現場會發生的事：有人把燈關暗、傳輸時壓得太狠、感測器增益調高帶進雜訊、輸送帶震動造成拖影、或者有人把相機往後挪了一點。結果非常集中：兩條路線都被同一件事打垮——雜訊。傳統管線從 0.23 個升到 2.4 個，偵測器從 1.17 個升到 9.0 個，其餘五種退化的影響都小得多。這裡有一個很值得記住的因果鏈。單純把亮度降到四成，兩邊幾乎都沒事（0.73 與 1.07 個）——第 17 章也量到同樣的事，模型對整體變暗相當不敏感。真正致命的不是暗，而是「為了應付暗而把感測器增益調高」所帶進來的雜訊。所以這張圖其實是圖 18-14 的延伸：把光打好不只是讓影像好看，它讓你不必調高增益，於是整條鏈都不必面對雜訊。兩條路線的差別則在別的地方：JPEG 壓到品質 15 時傳統管線幾乎無感（0.33 個），偵測器卻升到 2.0 個——壓縮瑕疵會讓影像偏離訓練分布（第 15 章），而傳統管線只關心前景與背景的對比還在不在。這也說明為什麼雙軌並行值得做：兩者的失效模式不同，同時出錯的機會比各自出錯低得多。對專題的意義在 M4 那一格：不要只在你蒐集資料的那個下午、那個位置測一次。把燈關掉一半測一次、把增益拉高測一次、把相機往後挪三十公分測一次。每一個你量到的掉幅，都是報告裡的「系統限制」那一段;而每一個你沒量到的，都會在評審面前自己出現。

圖 18-19　實驗室很準，現場為什麼崩

18.6.3 案例三：互動式手勢控制機器人

問題：讓人以自然的手勢指揮機器人，而非透過遙控器。做法：以 YOLO Pose 或手部關鍵點偵測取得手勢(第 16 章)→ 定義手勢語彙(如張開手掌代表停止、指向代表前進方向)→ 加入時序判斷避免誤觸發(第 12 章)→ 透過 ROS 2 發布控制指令給機器人(第 17 章)→ 在 Jetson 上即時執行。

這個案例最能體現 Play 的精神——手勢的設計本身就是一種創作，你可以玩出各種有趣的互動方式。工程重點在即時性(第 17 章的延遲預算)與可靠性(誤判會讓機器人做出非預期動作，須設計安全機制)。

18.6.4 案例四：低光影像增強比較研究

問題：夜間或室內昏暗環境下拍攝的影像品質不佳，哪種方法最有效?做法：自建一組含「暗處拍攝」與「充足光線參考」配對的影像資料集 → 分別實作與比較：Gamma 校正與 CLAHE(第 5 章)、多影格平均(第 5 章夜景模式)、維納濾波與非局部平均(第 7 章)、小波門檻化(第 11 章)、深度學習增強模型(第 15 章)→ 以 PSNR、SSIM(第 3 章)與主觀評分量化比較。

這是一個「研究型」而非「系統型」的專題，適合對演算法本身有興趣的同學。它的價值在於系統性的比較與誠實的數據呈現——這正是工程研究的核心素養，也直接對應 M3 里程碑要求的量化評估。

18.6.5 案例五：智慧鏡頭人臉辨識（樹莓派 5 + AI Camera）

情境：社團教室或實驗室的門禁，要求是即時、斷網也能運作，而且不把任何人的臉傳出去。這個題目之所以適合當期末專題，是因為它同時碰到本書三條主線：第 10、13 章的人臉偵測與特徵、第 15 章的嵌入向量，以及第 17 章的感測器上推論與隱私設計。

架構依 17.2.1 節的兩層切開。第一層在 AI Camera 的 IMX500 感測器裡執行人臉偵測，主機只拿到「有沒有人、框在哪裡」的張量後設資料;第二層只在偵測到人臉的那一瞬間才啟動——樹莓派取出那一小塊影像區域，算出嵌入向量，與已註冊的向量比對，成功就開門並記錄時間。請注意這個切法買到的東西：絕大多數的影格連解碼都沒有，而「是誰」這個高算力、低頻率的問題留給主機。這正是 17.1.2 節三層架構最乾淨的一個實例。

程式的骨架分成兩段。第一段在你的電腦上做，把模型轉成感測器的格式;第二段在樹莓派上做，讀取後設資料並接上比對流程。

```
# ── 在電腦上：把偵測模型轉成 IMX500 的部署格式 ──
from ultralytics import YOLO

model = YOLO("yolo11n.pt")        # 注意：目前支援到 YOLO11n 這一級
model.export(format="imx", data="coco8.yaml")
#   轉檔會自動做 INT8 量化（PTQ），產出 yolo11n_imx_model/
#   裡面的 packerOut.zip 就是要放到樹莓派上的部署包

# ── 在樹莓派 5 上：安裝韌體與工具 ──
#   sudo apt update && sudo apt full-upgrade
#   sudo apt install imx500-all
#   sudo reboot
```

上了樹莓派之後，重點是「只在需要時才讓影像進入主機」。以 Picamera2 為例，主迴圈大致是這個形狀：

```
from picamera2 import Picamera2

picam = Picamera2()
picam.start()

while True:
```

```
    # 感測器已經把推論做完了，這裡拿到的是「結果」而不是「影像」
    meta = picam.capture_metadata()
    faces = parse_detections(meta)        # 解析張量後設資料

    if not faces:
        continue                          # 絕大多數影格在這裡就結束了

    # 只有偵測到人臉時，才真的去取一張影像、而且只取框內那一塊
    frame = picam.capture_array()
    for (x1, y1, x2, y2) in faces:
        patch = frame[y1:y2, x1:x2]
        vec = embed(patch)                # 第 15 章的嵌入向量
        who, score = match(vec, registry) # 與已註冊的向量比對
        if score > THRESHOLD:
            open_door(who)
        else:
            notify_admin()
    del frame                             # 比對完即丟，不落地
```

這個案例的三個工程重點，剛好對應 M1 到 M5 要交代的事。一是模型的額度：人臉偵測模型必須塞進感測器上那很小的記憶體額度，而且只能是 INT8，所以這一層只做「有沒有臉」，不做「是誰」（17.2.1 節的三條硬規定）。二是驗收指標：門禁屬於「寧可擋錯也不要放錯」的場景，所以要報的是「冒名通過率」而不是整體準確率——承圖 18-16 的教訓，這個指標要在 M1 就寫進提案。三是樣本數：每個註冊者能拍到的臉通常只有幾十張，少數類別的樣本數才是瓶頸（圖 18-18 與推導 18.5.3）。

最後是倫理，而且這一題不能留到 M5。人臉是高度敏感的個人資料，所以圖 18-20 的三個問題必須在 M1 就回答，因為那時架構還改得動。具體的設計決策包括：公告蒐集範圍與目的、提供不使用人臉的替代方式（卡片或密碼）、只儲存比對用的特徵向量而非影像、設定保存期限並定期刪除、以及保留一條人工覆核與申訴的通道。請注意這些決定本身就是技術決定——「只存向量不存影像」這一條，直接決定了你的資料庫長什麼樣子。

18.6.6 案例六：開放語意的現場巡檢（Jetson Orin Nano Super + NanoOWL）

情境：一台巡檢機器人要在廠區或校園裡找出「這週要找的東西」，而那個清單一直在變——這週找沒戴安全帽的人，下週找倒地的推車，再下週找堆在逃生通道上的紙箱。用第 16 章的做法，每換一次清單就要標註與重訓一輪，以一學期的工時預算（推導 18.3.4）根本跟不上。這正是開放詞彙偵測（open-vocabulary detection）存在的理由：類別不再是訓練時焊死的，而是執行時用一段文字決定的。

架構：Jetson Orin Nano Super 作為主控 → 相機節點發布影像 → NanoOWL 節點依文字提示執行開放詞彙偵測 → 把結果發布成 ROS 2 訊息 → 決策節點記錄位置與時間並回報（第 17.5 節的節點與話題）。提示詞從設定檔讀入，現場就能改，不必回實驗室。NanoOWL 的作法是把 OWL-ViT 的影像編碼器用 TensorRT 編成引擎檔，所以第一次執行前要先在機器上建好引擎：

```
# ── 先建好 TensorRT 引擎（在 Jetson 上執行一次即可）──
#   python3 -m nanoowl.build_image_encoder_engine \
```

```
#       data/owl_image_encoder_patch32.engine

from nanoowl.owl_predictor import OwlPredictor

predictor = OwlPredictor(
    "google/owlvit-base-patch32",
    image_encoder_engine="data/owl_image_encoder_patch32.engine")

# 類別就是這一行字，改字就換了一組類別
PROMPT = ["a person without a helmet", "a cardboard box"]

output = predictor.predict(image=image, text=PROMPT, threshold=0.1)
for box, label, score in zip(output.boxes, output.labels,
output.scores):
    report(PROMPT[label], box, score)
```

NanoOWL 另外提供一個樹狀提示（tree prediction）的語法，可以把偵測與分類巢狀地組合起來，這在巡檢上很好用。方括號是「在這個東西裡面再找」，圓括號是「把它分成哪幾類」：

```
# 先找出每一張臉，再在每一張臉的區域裡找五官
#   --prompt="[a face [a nose, an eye, a mouth]]"

# 先找出每一個人，再判斷他有沒有戴安全帽
#   --prompt="[a person (wearing a helmet, not wearing a helmet)]"

# 把整張影像分類成室內或室外
#   --prompt="(indoors, outdoors)"
```

這個案例真正的教學價值在於它的限制，而不是它的炫目，而三個限制剛好都對應本章前面量過的東西。第一，提示詞是一個參數。模型比對的是那串字元產生的向量，不是你心裡的概念——單數與複數、加不加冠詞、換一個同義詞，結果可能天差地別。所以提示詞必須在開發集上調、在另一份從頭到尾不碰的測試集上報（圖 18-3 的開發集與測試集差距，在這裡會以更誇張的形式出現）。第二，它比同級的封閉詞彙模型重，因為每次推論都要同時處理影像與文字兩邊;這會直接吃掉第 17.4.2 節的延遲預算，所以巡檢機器人通常是「停下來看」而不是「邊走邊看」。第三，它在固定類別上的表現，通常不如一個專門為那些類別訓練過的偵測器。

務實的做法因此是把它當成第一道篩子，而不是最終的判斷。先用開放詞彙快速找出「可能有問題的畫面」，再交給人工複核、或交給一個針對該類別訓練過的小模型確認。等到某一個項目變成長期固定的檢查項（例如安全帽），就值得為它標註資料、訓練一個封閉詞彙的模型正式上線。這正是 18.3.1 節「最小可行版本」的精神：先把路走通，再把最爛的那一段換掉——而開放詞彙讓你在還沒有任何標註資料的第二週，就能先把路走通一次。

順帶一提，這個案例也示範了一個選題上的判斷。若你的題目要找的東西從一開始就固定（例如只數零件），那麼開放詞彙是多餘的，用第 16 章的做法又快又準;它的價值只在「清單會變」這件事上。所以在提案時要先問清楚：我的需求方到底會不會改清單?多久改一次?這個問題的答案，決定了你該不該走這條路。

18.7 全書技術整合地圖

走到這裡，你已經走完了從像素到系統的完整旅程。這張表回顧全書的四篇架構也是你規劃專題時的索引。

表 18-4 全書四篇架構與核心能力

也請記得那些貫穿全書、比任何單一技術更重要的思想：換一個域看問題(第 6 章)、多解析度與由粗到細(第 11、13、15 章)、投票對抗雜訊(第 9、13 章)、正則化對抗病態(第 7、14 章)、精確率與召回率的取捨(第 9、10、14、16 章)、以及「適合勝於最強」(第 9、10、11、13、16、17 章)。技術會更迭，這些思考方式不會。而本章用一個零件檢測站，把其中幾個判準第一次完整地量了出來：取像設計的槓桿最大（圖 18-14）、「數有幾個」與「長什麼樣子」難度差一個量級（圖 18-6、18-13）、資料的瓶頸是變異而不是張數（圖 18-7）、指標選錯會看到完全不同的世界（圖 18-16）、而最便宜的那條路往往在第二週就能跑起來（圖 18-9）。接下來的推導 18.7.1 會把這些收斂成一個可以照著跑的六步流程，作為你面對下一個新題目時的起手式。

18.7.1 推導：把全書的判準收斂成一個可以照做的流程

走到這裡，你手上有十八章的工具與六個反覆出現的想法。但真正的問題是：面對一個新的題目，第一步該做什麼?這一小節把全書的判準收斂成一個流程。它不是標準答案，而是一個可以照著跑一遍、並且每一步都有本章實測數字支撐的檢查順序。

第一步：先問取像端能不能控制。這一步排在最前面，是因為它的槓桿最大，而學生幾乎總是跳過它直接去挑模型。圖 18-14 的實測：同一個計數任務，穩定打光、乾淨背景、零件不相碰時，傳統管線與深度學習幾乎沒有差別，兩者都接近全對;把取像條件放掉，傳統管線直接崩掉。換句話說，一盞穩定的光源可能抵得過幾百張標註。若你的場域是產線、檢測站、實驗台——先把光與背景固定下來。若是果園、戶外、移動中的機器人——這個選項不存在，直接跳到第二步。

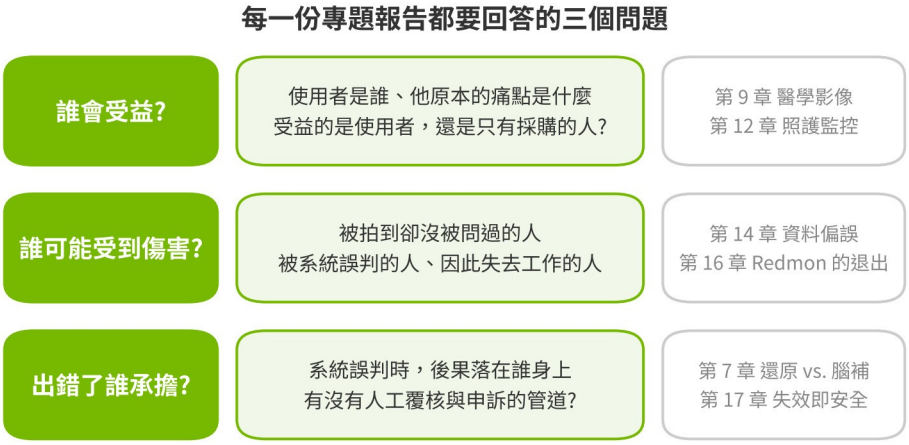
第二步：問系統要回答的是「有幾個、在哪裡」還是「長什麼樣子」。這兩件事的難度差了一個量級。圖 18-6 的實測：「找到零件」八張標註就有七成以上，而「判斷它壞了沒有」要幾十倍的資料才慢慢爬上來。圖 18-13 說得更直接：傳統管線能數，但不能判斷外觀;要用傳統方法做外觀判斷，就得再接上第 6 章的紋理分析或第 13、14 章的特徵加分類器，也就是再長出一整段管線。所以若你的題目同時需要這兩件事，請把它們切成兩個里程碑。

第三步：估資料的帳，而不是估模型的帳。圖 18-7 的實測結論很硬：同一批資料上多跑四倍的訓練步數，買到的進步遠小於把資料加到四倍。而圖 18-18 進一步說：要加的是少數類別。所以在 M1 的提案裡，資料那一欄要寫的是「我能拿到幾個瑕疵樣本」，而不是「我能拍幾張照片」。

第四步：定驗收指標，而且從使用者的痛點反推。圖 18-16 的實測：同一批模型，mAP@0.5 一路好看，而「實際的瑕疵裡抓到幾成」始終難看得多，計數誤差又是另一條曲線。三個指標、三種結論。接著用 18.5.3 節的式子反推樣本數：要宣稱的精度愈高，要測的張數愈多，而且是平方關係。

第五步：排延遲與硬體的帳。這一步直接沿用第 17 章：先確定任務允許的總延遲，再回頭分配給各環節，並且用第 99 百分位而不是平均值。圖 18-5 量過本章兩條路線的單張耗時差距約兩個數量級——這個數字決定你要不要 Jetson、要不要降解析度。

第六步：把失敗先想一遍。圖 18-19 列了六種「現場會發生的事」，每一種都在 M4 的實地測試裡量得出來。量到的每一個掉幅，都是報告裡「系統限制」那一段;沒量到的，會在評審面前自己出現。同時回答圖 18-20 的三個倫理問題——而且要在 M1 回答，因為那時架構還改得動。



這是一張檢核表，不是實測。18.5.2 節說它「不是加分題，而是基本功」，理由在右邊那一欄：這三個問題不是期末才冒出來的，它們從第 7 章就開始在這本書裡反覆出現。要讓這份檢核表不流於形式，有一個很有效的做法：把它放進 M1，而不是 M5。在還能改架構的時候問「誰可能受到傷害」，你可能會決定把人臉在端上就模糊掉、決定不儲存原始影像、或決定讓系統只輸出數量而不輸出身分——這些都是設計決策，到了 M5 就改不動了。這也呼應第 17 章的那句話：最有力的隱私保護不是承諾不看，而是技術上沒有東西可看。

圖 18-20　倫理三問：不是加分題

把六步連起來：控制取像 → 分清任務類型 → 估資料的帳 → 定指標與樣本數 → 排延遲預算 → 把失敗想一遍。你會發現這個順序裡，模型的選擇排得非常後面。這不是因為模型不重要，而是因為前面每一步都會改變「你需要什麼模型」這個問題的答案——甚至常常會讓答案變成「不需要模型」。這就是「適合勝於最強」在一張流程圖上的樣子，也是這門課十八章下來，最希望你帶走的那種判斷力。



這是全書的地圖，也是這門課的最後一張圖。上半部是四篇的結構：每一篇都建立在前一篇之上，而第四篇之所以能存在，是因為前三篇已經把影像整理成機器學得動的樣子。下半部那六個想法，才是叫獸真正希望你帶走的東西——因為函式會更新、模型會換代，但這些想法不會。當你在專題裡卡住時，不妨回頭問問它們：這個問題換一個域看會不會簡單一點?要不要先在粗解析度上找、再回到細解析度上確認?我是不是可以讓很多個弱證據投票，而不是相信一個強證據?我現在缺的到底是更強的模型，還是更適合的方法?

圖18-21　從像素到世界：全書的整合地圖

程式實作 Lab 18：期末專題

Lab 18-1(M1)提案與可行性評估

目標：找到一個你真心想做的題目。(a)腦力激盪：每人先獨立列出五個想做的題目，不要自我審查;(b)組內分享並票選，依「熱情」與「可行性」兩軸把候選題目畫在座標圖上;(c)對選定的題目做調查：是否已有類似作品?他們怎麼做?你的切入點是什麼(d)畫出系統架構草圖：輸入是什麼、經過哪些處理、輸出什麼、用什麼硬體;(e)定義最小可行版本與延伸目標;(f)產出提案簡報，包含問題描述、目標、方法、時程與分工。

Lab 18-2(M2)資料蒐集與原型

目標：及早驗證核心技術可行。(a)蒐集資料：注意第 14 章的教訓——不同類別要在相同條件下拍攝，避免模型學到「捷徑」;記錄拍攝條件;(b)標註並依第 14 章原則切分訓練/驗證/測試集;(c)做出最小可行原型，哪怕準確率很低、介面很醜——重點是「能從頭跑到尾」;(d)記錄第一次的量化結果作為基準線;(e)反思：實際做過之後，原本的規劃有哪些需要調整?

Lab 18-3(M3-M4)核心開發與整合部署

目標：把原型變成可用的系統。(a)迭代改進核心演算法，每次改動都記錄參數與結果(工程筆記);(b)量化評估：依任務選擇適當指標(準確率、mAP、PSNR 等)，並誠實記錄失敗案例;(c)系統整合：把各模組串接，進行端到端測試;(d)效能最佳化與部署

(第 17 章)：量測延遲與 FPS，依需要最佳化;(e)實地測試：把系統帶到真實環境(果園、實驗室、走廊)測試，記錄與實驗室結果的差異——這往往是最有價值的學習;(f)依測試結果再迭代一輪。

Lab 18-4(M5)展示、文件與反思

目標：完整交付並沉澱學習。(a)撰寫技術文件：系統架構、環境安裝、使用說明、程式碼結構——標準是「另一個人照著做能重現你的成果」;(b)準備展示：依 18.5.1 節的五個步驟編排，務必有現場 demo 或實測影片;(c)誠實呈現限制與失敗案例;(d)回答三個倫理問題：誰受益、誰可能受害、出錯了誰承擔;(e)個人反思：這學期你學到最重要的一件事是什麼?若重來一次會怎麼做?接下來想繼續學什麼?(f)把成果整理成作品集——這是你日後求職、升學最有力的證明。

Lab 18-5 建立你自己的基準線與學習曲線（M2 的量化版）

目標：把 M2 從「做出一個能跑的東西」升級成「做出一個能回答問題的東西」。本 Lab 的每一步都對應本章的一張實測圖，你要在自己的題目上重做一遍。(a)最小可行版本：用不需要訓練的方法（門檻、色彩、影格差分、樣板比對，視你的題目而定）做出第一個端到端版本，並量出它的基準線數字與單張耗時（對應圖 18-9）;(b)調參一輪：選兩個最關鍵的參數做網格掃描，畫出你自己的參數地形圖，記錄最佳組合、中位數與最差值（對應圖 18-11）;注意要在開發集上挑，在另一份測試集上報;(c)學習曲線：若你的題目需要訓練，從少量標註開始（例如 8、16、32、64 張），每一次都給相同的梯度步數，畫出你自己的學習曲線（對應圖 18-6）;(d)判斷瓶頸：曲線若還在上升，繼續標註;若已經打平，改做變異實驗——換一批不同光線／角度／位置的影像再訓練一次，比較結果（對應圖 18-7）;(e)寫下結論：以一頁說明「要達到驗收標準，我還需要什麼」，並據此修正 M3 到 M5 的排程。討論：若 (a) 的最小可行版本已經達到驗收標準，你會怎麼做?請注意這是好消息而不是壞消息——把省下來的時間花在穩健性與使用者介面上，價值遠高於硬把一個深度模型塞進去。

Lab 18-6 驗收與交付：讓你的數字站得住腳

目標：把 M5 從「做一場簡報」升級成「交付一個可被檢驗的系統」。(a)樣本數反推：先寫下你要宣稱的精度（例如「漏檢率低於 5%」），用推導 18.5.3 的標準誤或三倍法則算出需要測幾張，並誠實檢查你手上的測試集夠不夠大、少數類別有幾個樣本;(b)鎖定測試集：確認測試集從 M2 之後就沒有被用來挑過任何東西，並檢查三種常見的外洩（增強後才切分、連續影格被打散、用測試集挑模型，見圖 18-17）;(c)逐類別報告：列出每一類的精確率、召回率與樣本數，不要只報平均（對應圖 18-18）;(d)退化測試：設計至少五種你的現場真的會發生的退化（光線、雜訊、壓縮、模糊、距離、遮擋……），各測一次並製成表（對應圖 18-19）;每一個掉幅都要寫進報告的「系統限制」那一段;(e)失效設計：為最嚴重的那兩種退化設計偵測與因應方式（例如影像品質檢查、信心分數監控、人工覆核觸發條件，承第 17 章的失效即安全）;(f)倫理三問：書面回答誰受益、誰可能受害、出錯了誰承擔，並指出你在架構上做了哪些對應的設計決策（對應圖 18-20）;(g)可重現性：整理出一份文件，標準是「另一個人照著做能重現你的數字」——包含資料如何切分、參數的完整設定、環境與版本。討

論：(d) 裡若有某一種退化讓系統完全失效，而你已經沒有時間修了，該怎麼辦?請思考為什麼「在報告裡誠實寫出來並說明因應方式」比「不提它」更專業，也更安全。

本章重點整理

- 4P 創意學習：Projects(在創造中學習)、Passion(投入真心在乎的題目)、Peers(協作與回饋)、Play(勇於嘗試、不怕失敗);源自 MIT Media Lab 的終身幼兒園研究群。
- 創意學習螺旋：想像、創造、遊玩、分享、反思，循環迭代且每繞一圈都站得更高;對應專題的多次迭代與里程碑規劃。
- 選題落在「熱情」與「可行性」的交集;可行性須誠實評估時間、資料、硬體與技術落差，並設定最小可行版本以確保底線。
- 里程碑 M1 提案、M2 資料與原型、M3 核心功能、M4 整合部署、M5 展示反思;每個里程碑都要有可展示的成果，並儘早做出粗糙原型以及早發現困難。
- 團隊實務：用 Git 版本管理、依模組分工但約定好介面並儘早整合、撰寫工程筆記記錄實驗參數與結果。
- 展示應從問題出發、盡早展示成果、說明關鍵取捨、誠實呈現限制、並反思展望;評量對應 4P，涵蓋技術、問題意識、協作、迭代、溝通與倫理反思。

習題

1. 請說明 4P 各自的意義。為什麼說「熱情」是其中最容易被忽略卻最關鍵的一項?

   提示與參考解答：Projects：在動手創造具體事物的過程中學得最深。Passion：投入真心在乎的題目。Peers：與同伴協作、互相回饋。Play：願意嘗試、不怕失敗的實驗心態。熱情最關鍵是因為它決定了你願意投入多少時間、承受多少挫折——而學習的深度往往正比於投入;它也最易被忽略，因為傳統作業多是老師指定題目，學生沒有選擇的餘地。

2. 承叫獸開講：Resnick 為什麼認為「幼兒園」是千年來最重要的發明?他對學校教育的主張是什麼?

   提示與參考解答：因為幼兒園是教育史上的根本創新——不是把知識塞給孩子，而是讓孩子在動手做、玩耍、與同伴互動中自己建構理解，這是創造力的搖籃。他的主張：不該讓幼兒園變得像其他學校(近年幼兒園小學化，積木被練習卷取代)，而該讓其他學校、甚至整個人生變得更像幼兒園——保留那套學習方式的精髓(即 4P)。

3. 請說明創意學習螺旋的五個階段。為什麼稱為「螺旋」而非「循環」?

   提示與參考解答：五階段：想像(產生想法)、創造(動手做出來)、遊玩(測試試探)、分享(給人看、聽回饋)、反思(想想哪裡好壞與為什麼)，然後回到想像。稱為螺旋是因為每繞一圈都回到起點但站在更高位置——想法更成熟、技術更純熟、理解更深。它提醒我們專題不應期待一次做對，而要規劃多次迭代。

4. 好的專題題目應落在哪兩個條件的交集?若只有其中之一會發生什麼?

   提示與參考解答：落在「熱情(你真心想做)」與「可行性(時間與資源內做得出來)」的交集。只有熱情而不可行：會做到一半崩潰、無法交付。只有可行而無熱情：會

做得索然無味、成果平庸、學習有限。可行性須誠實評估時間、資料、硬體、技術落差四項。

5. 什麼是「最小可行版本」?為什麼 M2 階段就要做出粗糙原型，而不是先完整規劃?

   提示與參考解答：最小可行版本是一個「一定做得出來」的核心功能，確保專題有底線，行有餘力再往上加。及早做粗糙原型的理由：許多專題失敗是因為把太多時間花在規劃準備，後期才發現核心技術行不通。先做能從頭跑到尾的版本(哪怕準確率很低)，就能及早發現真正的困難點並調整方向——這是遊玩精神的工程版本(快速原型)。

6. 團隊專題有哪三項工程實務建議?為什麼「太晚整合」是常見的災難?

   提示與參考解答：一、用 Git 版本管理(不只備份，更讓你安心嘗試、多人平行開發、回顧修改理由)。二、依模組分工但務必約定介面(輸入輸出格式)並儘早整合測試。三、撰寫工程筆記記錄每次實驗的參數、結果與觀察。太晚整合的災難：各模組單獨都能跑，合起來卻因介面不符、時序問題、環境衝突而全面卡關，而此時已無時間修正。

7. 一場好的專題展示建議依什麼順序安排?為什麼「誠實呈現限制」不會扣分?

   提示與參考解答：順序：一、從問題出發(為什麼要做，用具體情境抓住聽眾);二、盡早展示成果(demo 或實測影片最有說服力);三、說明方法(重點在關鍵決策與取捨，而非流水帳);四、誠實呈現限制(何時會失敗、目前準確率、還有什麼沒做到);五、反思與展望。誠實不扣分，反而顯示你真正理解自己的系統——這也是第 7、16 章工程倫理的體現;隱藏限制的系統反而危險。

8. 在選擇專題技術時，為什麼「不要為了用最新技術而選題」?請以本書的判準說明。

   提示與參考解答：因為工程的價值在於解決問題，不在炫技。承第 9、10、11、13、16、17 章反覆強調的「適合勝於最強」：一個用 Otsu 加連通元件就能穩定運作的零件計數系統，價值不亞於用上最新模型卻不穩定的系統;受控環境用傳統方法可能又快又省又可解釋，複雜開放場景才需要深度學習。應為了解決問題而選技術。

9. 你的專題報告中應回答哪三個倫理問題?請以本書各章的倫理線索說明為什麼這是「基本功」而非加分題。

   提示與參考解答：三問題：誰會受益?誰可能受到傷害?如果它出錯了，後果由誰承擔?這是基本功的理由：第 7 章「還原 vs. 腦補」提醒區分找回與編造的資訊、第 9 章醫學影像須醫師確認、第 12 章照護監控涉及隱私尊嚴、第 14 章資料偏誤讓模型學到捷徑、第 16 章 Redmon 因軍事與隱私疑慮退出、第 17 章端上處理保護隱私——技術能力愈強，對後果的想像力就愈是專業素養不可分割的一部分。

10. 請從表 18-2 選一個專題方向，列出你會用到本書哪些章節的技術，並說明各自負責什麼環節。

    提示與參考解答：本題為開放題，評分重點在技術鏈的完整與合理。範例(果園產量估計)：第 4 章相機校正(像素換算實際尺寸以估果實大小)、第 5 章 CLAHE 與白平衡(戶外光線變化前處理)、第 8 章色彩分析(成熟度判斷)、第 14 章資料切分與評估、第 15 章遷移學習與資料增強(樣本不足)、第 16 章 YOLO(果實偵測計數)、第 17 章邊緣部署(田間無網路須機上運算)。

11. 請說出三個貫穿本書、比任何單一技術更重要的思想，並各舉兩章說明它如何反覆出現。

    提示與參考解答：可任選三個：一、換一個域看問題(第 6 章頻率域解週期雜訊、第 11 章小波補位置資訊)。二、多解析度由粗到細(第 11 章小波金字塔、第 13 章 SIFT 尺度空間、第 15 章 CNN 層級特徵)。三、投票對抗雜訊(第 9 章霍夫轉換、第 13 章 RANSAC)。四、正則化對抗病態(第 7 章維納濾波、第 14 章對抗過度擬合)。五、精確率與召回率的取捨(第 9 章 Canny 雙門檻、第 10 章 Viola-Jones 參數、第 14、16 章評估指標)。六、適合勝於最強(第 10 章的 Haar 級聯、第 11 章 JPEG 勝過 JPEG 2000、第 17 章邊緣模型選擇)。

12. (反思題)請回顧本學期：你學到最重要的一件事是什麼?哪一章的內容最出乎你意料?接下來你想繼續學什麼?請具體說明並規劃你的下一步。

    提示與參考解答：本題無標準答案，重點在反思的具體與真誠。好的回答應：指出具體的學習事件(而非泛泛而談「學到很多」)、說明它為什麼改變了你的理解、並提出可執行的下一步(如深入某個主題、參加競賽、做開源貢獻、修習進階課程如 3D 視覺/SLAM/生成式模型、或把專題延伸為更完整的作品)。

13. 為什麼 18.3.2 節堅持「每個里程碑都要有可展示的成果」？「這兩週我讀了很多論文」為什麼不算一個里程碑?

    提示與參考解答：因為創意學習螺旋的五個階段裡，「遊玩、分享、反思」這三個都需要有一個能動的東西才跑得起來——沒有東西可以玩，就沒有東西可以反思，於是整個螺旋停在「想像」與「創造」之間空轉。圖 18-3 把這件事量了出來：那五輪參數調整，每一輪的改進都來自上一輪量到的結果，而不是來自更多的思考。沒有第 0 輪那個難看的數字（測試集 8%），就不會有第 1 輪的方向。讀論文當然必要，但它是「創造」的輸入，不是螺旋的一圈。一個可操作的判準：里程碑的交付物要能讓別人問出「它在什麼情況下會失敗」這個問題。進度報告沒辦法被這樣問，一個會動但很爛的原型可以。

14. 圖 18-3 的五輪調參裡，哪一輪帶來了幾乎全部的進步?這對「卡住的時候該做什麼」有什麼啟示?

    提示與參考解答：第 1 輪，也就是把全域門檻換成自適應門檻那一次——測試集從 8% 跳到 83%，後面四輪加起來只再移動了幾個百分點。啟示是：進步的分布通常極不平均，而大的那一步來自「換方法」，不是「換數字」。這裡的原因可以說得很具體：全域門檻假設整張影像的亮度一致，而這批影像有斜向打光與鏡頭暗角，這個假設根本不成立;再怎麼微調那個門檻值，都救不了一個錯的假設。所以卡住時該先問的是「我這一步的假設對不對」，而不是「這個參數再調大一點會不會好一些」。一個實用的習慣：每次調參前先寫下「我預期它會怎麼變、為什麼」，若結果和預期不符，那通常表示你對系統的理解有誤——而那個誤解才是真正要修的東西。

15. 你的系統在 20 張測試影像上全部正確。可以在報告裡寫「準確率 100%」嗎?請用推導 18.5.3 的式子說明，並算出要宣稱「失敗率低於 1%」需要測幾張。

    提示與參考解答：不可以。比例的標準誤是 $SE = \sqrt{p(1-p)/n}$，但 $\hat{p} = 1$ 時這條式子給出 0，顯然不合理;這時要用三倍法則：n 次全對時，真正的失敗率上限約是 3/n。n = 20 時上限是 15%——所以「20 張全對」真正的意思是「有九成五的把握失敗率低於 15%」，離 100% 很遠。要宣稱失敗率低於 1%，需要 $3/n < 0.01$，即 $n > 300$ 張。而且這 300 張必須是模型沒看過的（圖 18-17），也必須彼此獨立——若它們是同一段影片的連續影格，等效樣本數遠小於 300。順帶算一個常見的情況：測 60 張、量到九成，$SE = \sqrt{0.9 \times 0.1/60} \approx 0.039$，也就是三到四個百分點;這正

是圖18-3 裡93% 與90% 分不出高下的原因。報告的寫法：「在120 張未見影像上mAP@0.5 為86%」，而不是「準確率86%」。

16. 圖 18-6 的學習曲線在 32 張之後就打平了。這是不是表示「32 張就夠了」?請說明打平真正的意思。

    提示與參考解答：不是。打平的意思是「這批資料的變異用完了」，不是「資料夠了」。該實驗的影像全部由同一個生成器產出：同一種零件、同一個尺寸範圍、同樣那幾種瑕疵、同一種退化方式。第33 張和第3 張長得太像，所以模型學不到新東西。圖18-7 右半把這件事證明了：128 張理想取像條件下的影像，在現場影像上只有66.9% 的mAP;而32 張真正的現場影像有77.6%——四分之一的張數，高出10.7 個百分點，瑕疵類更高出16.9 個百分點。所以判斷「資料夠不夠」的正確問法不是數張數，而是列情況：我涵蓋了幾種光線、幾種背景、幾種角度、幾種距離、每一種瑕疵型態各有幾個樣本?任何一格是零，那一格就是你的系統將來會出錯的地方。實務上還有一個訊號可以用：若把訓練集加倍而驗證分數不動，下一步該做的是去新的情境蒐集資料，而不是繼續在原地加量。

17. 學習曲線打平之後，加算力有用嗎?請依圖 18-7 的實測說明，並指出真正有效的那個變因是什麼。

    提示與參考解答：幾乎沒用。圖18-7 左半實測：32 張基準步數是mAP 83.7%／瑕疵74.3%;同一批32 張跑四倍的梯度步數只動到84.2%／74.0%;張數加到128 張是81.3%／69.8%。三者在量測噪聲範圍內，算力與張數都已經不是瓶頸。真正有效的變因是資料涵蓋的「情境變異」：把訓練影像換成與使用現場相同條件的影像，mAP 從66.9% 升到77.6%。道理很單純——模型學得到的只有你給它看過的變異。在實驗室拍一千張，它就只學會實驗室。這也是18.3.1 節「在真實環境蒐集訓練資料」那句話的量化版本，以及第15 章分布偏移在專題尺度上的樣子。對排程的意義：資料蒐集要在M2 就去現場做一次，不要等到M4 整合時才發現訓練集和現場是兩回事——那時已經沒有時間重新標註了。

18. 請用推導 18.3.4 的方法，為一個四人小組估算一學期可用於開發的工時，並說明為什麼「沒有重做的空間」會決定專題的排程順序。

    提示與參考解答：總工時 = 人數 × 週數 × 每人每週時數。以 4 人 × 15 週 × 每週 3 小時計算是180 人時。扣掉協作（開會、同步、合併程式）約15%、環境與工具約10%、文件與展示約15%，實際可用於開發的約180 × 0.6 = 108 人時。把圖18-5 的實測填進去：本章這個零件任務每張平均14.1 個框，以每框四秒估算，標256 張約4 小時;每次訓練5 到7 分鐘，而你不會只訓練一次。再加上資料蒐集、核心開發、系統整合（圖18-10 說明了為什麼不能省）、實地測試（圖18-19 的六格），108 人時就滿了。關鍵在於：這個預算裡沒有任何一格留給「重做」。既然不能重做，唯一降低風險的方式就是讓「發現做不到」儘早發生，所以最不確定的那一段要排在最前面。這與直覺相反——大家習慣先做有把握的部分（介面、架構圖、簡報），因為那看起來有進度，但有進度和降低風險是兩件事。

19. 為什麼 M2 要先做一個「粗糙但能從頭跑到尾」的版本?請以圖 18-9 的實測說明它的投資報酬率。

    提示與參考解答：圖18-9 比了三條路線在同一批現場影像上的計數表現。照抄預設值的傳統管線：零準備、寫完就能跑，平均數錯3.20 個。同一條管線花約半天調過參數：誤差降到0.28 個。標註256 張再訓練的偵測器：準備成本約4.1 小時，誤差1.20 個。也就是說，在「數有幾個」這件事上，半天的調參打敗了四小時的標註加訓練，而且快了約二十倍。要公平地補充：偵測器在此只拿到很小的訓練預算，信心門檻也只在開發集上粗調過，給它更多資源它會更好——而且它能做傳統管線

做不到的事（圖 18-13 的第二格）。但這正是 M2 的論點：在你還沒有資源的那兩週，最小可行版本幾乎免費，卻立刻告訴你三件事——這個任務的基準線在哪裡、它到底難在哪裡、以及那段差距值不值得用幾十小時去填。很多時候答案是不值得，而那就是你省下來的一整個學期。

20. 圖 18-10 的四種錯誤都不是程式寫錯，而是模組之間的約定不一致。請說明它們為什麼特別難抓，以及在 M1 與 M2 分別該做什麼來預防。

    提示與參考解答：四種錯誤的實測（計數平均絕對誤差，越低越好）：介面都對 1.50 個;把 BGR 當成 RGB 傳進去 3.13 個;數值範圍談錯 2.93 個;輸入尺寸談錯（先縮到 96×96 再交出來）2.53 個。另外還有一種數量完全正確、位置卻全錯的：偵測模組輸出 xyxy、下游按 xywh 解讀，框的平均 IoU 只剩 0.04。難抓的原因有三：一、每個模組單獨測試都是對的，只有合起來才錯;二、不會丟出例外，系統只會安靜地算錯，所以團隊往往花好幾天在調模型，從沒懷疑過介面;三、症狀是「數字有點難看」，而數字難看有一百種可能的解釋。M1 該做的：把每個模組的輸入輸出寫成白紙黑字——格式、通道順序、數值範圍、座標系、單位、影像尺寸。M2 該做的：跑一次端到端，哪怕每個模組裡面都還是假的（回傳固定值也行）。這兩件事加起來花不到半天，卻擋掉團隊專題最常見的災難。

21. 圖 18-11 掃過 72 種參數組合，最好的一格是 91.7%，中位數只有 27.8%。這對「調參」與「工程筆記」分別說明了什麼?另外，為什麼面積下限調得很小並沒有被懲罰?

    提示與參考解答：對調參而言，它說明參數空間的好區域很小——72 格裡只有 4 格落在最佳值的五個百分點之內，所以隨便試幾個值就宣稱「調過了」是沒有意義的。對工程筆記而言，它說明你不可能記得住：三個月後要回答「那次 91.7% 是怎麼跑出來的」，只有紀錄救得了你，而這張表本身就是期末報告最有價值的素材。至於面積下限：它調得很小卻沒有被懲罰，是因為前一步的開運算已經把雜訊點清掉了，所以沒有小面積的假目標需要靠面積下限來濾。這是參數之間的交互作用——單獨調一個參數得到的結論，換一個前處理就可能不成立。因此筆記要記的是「整組設定」，而不是單一數字;也因此報告裡寫「amin 設 200 最好」是沒有意義的，要寫「在自適應門檻 blockSize=91、開運算核 5×5 的前提下，amin 設 200 最好」。

22. 有人說「現在都用深度學習了，打光沒那麼重要」。請用圖 18-14 的實測回應這個說法。

    提示與參考解答：實測結果（計數平均絕對誤差，越低越好）：取像設計做好時，連照抄預設值的傳統管線都是 0.00 個;取像將就著用，同一段程式錯到 2.80 個。要把它救回來，你可以花半天調參數（降到 0.25 個）或花幾小時標註與訓練（0.90 個）——兩者都有效，但都沒有一開始就把光打好來得好，而且都貴得多。所以那個說法是倒果為因的：深度學習讓你「在無法控制取像時還有辦法」，而不是讓「控制取像變得不重要」。一盞穩定的光源、一塊素色背板、一個固定的工作距離，可能比多標五百張影像更有效，而且便宜得多。這與第 7、8、17 章一再強調的是同一件事：很多影像處理問題的最佳解不在演算法裡，而在取像那一端。當然要誠實：果園、戶外、移動中的機器人沒有這個選項，那時標註的錢就必須花——所以專題的第一個決定不是選模型，而是問「我的現場屬於哪一種」。

23. 圖 18-16 顯示 mAP@0.5 一路在 78% 到 84% 之間，瑕疵類召回率卻只有 55% 到 62%。這個差距從哪裡來?期末報告該報哪一個?

    提示與參考解答：差距來自兩件事。一、mAP 是兩個類別的平均，而良品那一類的 AP 有 93.2%，把平均往上拉（圖 18-18：平均 83.2% 離良品很近、離瑕疵很遠）。二、mAP 是對所有信心門檻積分出來的面積，而召回率是在一個實際工作點上的數

字;前者描述「這個模型的潛力」，後者描述「你上線那天會發生什麼」。該報哪一個?報使用者在乎的那一個。對一條產線，要簽字負責的是「流出去的不良品有多少」，也就是瑕疵召回率;對一個計數應用，要看的是計數誤差（圖18-16 右，而且它甚至不是單調的）。三個指標、三種結論，高低順序還會隨資料量變來變去。所以驗收指標必須在M1 的提案裡就寫清楚，而且要從使用者的痛點反推，不是從你會算的東西挑。一個mAP 八成、卻漏掉四成不良品的系統，若報告只寫那個八成，不算說謊，但也不算誠實。

24. 請說明圖 18-18 的類別不平衡與圖 18-17 的資料外洩，各自會讓你的驗收數字偏離真相多少，以及它們共同的對策是什麼。

提示與參考解答：類別不平衡（圖18-18）：訓練集有2842 個良品、643 個瑕疵，瑕疵只佔18.5%。結果是良品AP 93.2%、瑕疵AP 73.3%，而兩類平均83.2% 看起來還行;召回率的差距更大：99.6% 對60.0%。偏離的方向是「樂觀」——平均值把你最在乎的那一類藏了起來。資料外洩（圖18-17）：故意讓20 張驗證影像也進訓練集，mAP 量到88.3%、瑕疵類80.2%;同一個模型在真正沒看過的影像上是81.0% 與69.9%，分別掉了7.3 與10.3 個百分點。這些百分點憑空出現，而且不會發出任何警告。共同的對策有三：一、永遠報逐類別的數字與樣本數，不要只報平均;二、在M2 就把資料切成三份並寫進文件，測試集鎖起來，只在M5 開一次;三、注意等效樣本數——同一支影片的連續影格、同一張影像的增強版本，都不算獨立的樣本（推導18.5.3）。

25. （綜合設計題）請用推導 18.7.1 的六步流程，為你自己的專題題目完整走一遍，每一步都要寫出具體的答案與理由，並指出你目前最大的風險在哪一步。

提示與參考解答：本題為開放設計題，評分重點在六步是否都有具體答案（而非抽象描述），以及風險判斷是否誠實。六步與各自的檢查點——一、取像能不能控制?若能，先把光、背景、工作距離固定下來，並重測一次最簡單的方法（圖18-14 顯示這一步的槓桿最大）。二、系統要回答「有幾個、在哪裡」還是「長什麼樣子」?兩者難度差一個量級（圖18-6），同時需要就切成兩個里程碑。三、資料的帳：我能拿到幾個少數類別的樣本、涵蓋幾種情境?注意要的是變異不是張數（圖18-7）。四、驗收指標是什麼、要測幾張?從使用者痛點反推指標（圖18-16），再用三倍法則或標準誤反推樣本數（推導18.5.3）。五、延遲與硬體的帳：總延遲上限多少、分配給哪些環節、用p99 而非平均（第17 章）。六、失敗與倫理：列出會發生的退化並各測一次（圖18-19）、回答誰受益／誰受害／誰承擔（圖18-20），而且要在M1 回答。好的答案會指出一個具體的風險並附上驗證計畫，例如：「最大的風險在第三步——我只能在白天的溫室拍到資料，但系統要在清晨運作;所以M2 的第一件事是去清晨拍30 張，先確認模型在那個光線下還能不能用。」注意這個流程裡「選哪個模型」排得非常後面——因為前面每一步都會改變這個問題的答案，甚至常常讓答案變成「不需要模型」。

結語：從像素到世界

十八章的旅程到這裡告一段落。回頭看，我們從一張影像最基本的像素開始，學會了如何強化它、還原它、分析它的頻率、分割出物件、萃取特徵，再一路走到讓機器自己學習、自己辨識，最後把這一切裝進機器人裡，讓它能在真實世界中看見與行動。

但叫獸真正希望你帶走的，不是那些函式的用法——那些查得到、也會更新。我希望你帶走的是三樣東西。第一是判斷力：面對一個問題，知道有哪些工具可選、各自的代價是什麼、什麼情況下該用哪一個。這門課花了大量篇幅在比較與取捨，就是

為了培養這件事。第二是連結的眼光：當你發現卷積、量化、多尺度、投票、正則化這些概念在不同章節中以不同面貌反覆出現時，你就開始像一個真正的工程師那樣思考了——看見表象之下的共通結構。第三是責任感：每一章的故事裡都有人在做選擇，而技術愈強大，選擇就愈重要。

最後，請記得叫獸開講裡那些人。他們不是教科書上的名字，而是一個個在自己的處境裡，帶著熱情、好奇與不甘，把不可能變成可能的人。傅立葉的論文被退了稿，他繼續改;哈伯望遠鏡近乎失明，工程師用數學把影像救回來;一群創客翻譯著沒人看得懂的中文文件，意外打開了物聯網的大門。他們做的事，和你在期末專題裡要做的事，本質上是同一件事——只是規模不同而已。

所以，去做一個你真心想做的東西吧。和你的夥伴一起，帶著玩的心情，允許自己失敗、允許自己重來。這門課結束了，但那個螺旋不會停——想像、創造、遊玩、分享、反思，然後再一次想像。

祝你玩得開心。我們在你的作品裡見。

——機器人叫獸 李世淵

參考資料與延伸閱讀

- Resnick, M., Lifelong Kindergarten: Cultivating Creativity through Projects, Passion, Peers, and Play, MIT Press, 2017.(本章叫獸開講之史料來源;2018 年 PROSE 教育實務獎)
- MIT Media Lab, Lifelong Kindergarten Group:https://www.media.mit.edu/groups/lifelong-kindergarten
- Papert, S., Mindstorms: Children, Computers, and Powerful Ideas, Basic Books, 1980.(建構主義學習經典，Resnick 的師承)
- Scratch 程式學習平台：https://scratch.mit.edu
- 本書第 1 至 17 章之各章參考資料。
- 補充教材投影片下載：https://page.line.me/prof(機器人叫獸)